

داده کاوی

دانشگاه آزاد شیراز - دانشکده مهندسی کامپیوتر

ترم اول سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل هفتم : Classification: advanced methods

استاد: دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی): حمید نامجو و امیرحسین همتی

فهرست

- 7 Classification: advanced methods
 - 7.1 Feature selection and engineering
 - 7.1.1 Filter methods
 - 7.1.2 Wrapper methods
 - 7.1.3 Embedded methods
 - 7.2 Bayesian belief networks
 - 7.2.1 Concepts and mechanisms
 - 7.2.2 Training Bayesian belief networks
 - 7.3 Support vector machines
 - 7.3.1 Linear support vector machines
 - 7.3.2 Nonlinear support vector machines
 - 7.4 Rule-based and pattern-based classification
 - 7.4.1 Using IF-THEN rules for classification
 - 7.4.2 Rule extraction from a decision tree
 - 7.4.3 Rule induction using a sequential covering algorithm
 - 7.4.4 Associative classification
 - 7.4.5 Discriminative frequent pattern-based classification
 - 7.5 Classification with weak supervision
 - 7.5.1 Semisupervised classification
 - 7.5.2 Active learning
 - 7.5.3 Transfer learning
 - 7.5.4 Distant supervision
 - 7.5.5 Zero-shot learning
 - 7.6 Classification with rich data type
 - 7.6.1 Stream data classification
 - 7.6.2 Sequence classification
 - 7.6.3 Graph data classification
 - 7.7 Potpourri: other related techniques
 - 7.7.1 Multiclass classification
 - 7.7.2 Distance metric learning
 - 7.7.3 Interpretability of classification
 - 7.7.4 Genetic algorithms
 - 7.7.5 Reinforcement learning

Section 7.1

7.1.1. Filter Methods

انتخاب ویژگی (Feature Selection)

انتخاب ویژگی (Feature Selection) گامی حیاتی در فاز پیش پردازش داده است که شامل شناسایی و انتخاب مرتبطترین ویژگی‌ها برای یک وظیفه خاص می‌شود. روش‌های فیلتر

(Filter Methods) دسته‌ای از تکنیک‌های انتخاب ویژگی هستند که ارتباط ویژگی‌ها را بر اساس معیارهای آماری یا معیارهای امتیازدهی خاصی ارزیابی می‌کنند. این روش‌ها ویژگی‌های هر ویژگی را مستقل از مدل یادگیری ماشین (Machine Learning Model) می‌سنجند.

نکات کلیدی:

- **استقلال:** روش‌های فیلتر ارتباط هر ویژگی را مستقل از سایر ویژگی‌ها ارزیابی می‌کنند.
- **معیارهای امتیازدهی:** ویژگی‌ها بر اساس معیارهای آماری مانند همبستگی (Correlation)، اطلاعات متقابل (Mutual Information) یا آزمون‌های آماری رتبه‌بندی یا امتیازدهی می‌شوند.
- **پیش‌پردازش:** روش‌های فیلتر به عنوان یک مرحله پیش‌پردازش قبل از آموزش یک مدل یادگیری ماشین (Machine Learning Model) اعمال می‌شوند.
- **آستانه انتخاب:** یک آستانه برای انتخاب ویژگی‌های با رتبه برتر تعیین می‌شود و بقیه کنار گذاشته می‌شوند.
- **مزایا:** از نظر محاسباتی کارآمد هستند و می‌توانند داده‌های با ابعاد بالا (High-dimensional Data) را مدیریت کنند. این روش‌ها مستقل از مدل (Model-agnostic) هستند که آن‌ها را برای الگوریتم‌های مختلف مناسب می‌سازد.

مثال عملی در پایتون (Python):

یک مثال عملی با استفاده از مجموعه داده «سرطان سینه ویسکانسین» (Breast Cancer Wisconsin Dataset) را در نظر بگیریم. ما از آزمون آماری کای‌دو (χ^2) به عنوان یک روش فیلتر برای انتخاب مرتبط‌ترین ویژگی‌ها استفاده خواهیم کرد.

```
In [ ]: from sklearn import datasets
from sklearn.feature_selection import SelectKBest, chi2
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# بارگذاری مجموعه داده سرطان پستان ویسکانسین
cancer = datasets.load_breast_cancer()
X = cancer.data
y = cancer.target

# تقسیم داده‌ها به مجموعه آموزشی و آزمون
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# اعمال انتخاب ویژگی با استفاده از آزمون کای-اسکوئر (Chi-squared)
k_best_selector = SelectKBest(score_func=chi2, k=10)
X_train_selected = k_best_selector.fit_transform(X_train, y_train)
X_test_selected = k_best_selector.transform(X_test)

# ایجاد یک طبقه‌بند جنگل تصادفی با 100 درخت
random_forest_classifier = RandomForestClassifier(n_estimators=100, random_state=42)

# آموزش طبقه‌بند روی ویژگی‌های انتخاب‌شده
random_forest_classifier.fit(X_train_selected, y_train)

# انجام پیش‌بینی روی داده‌های آزمون
predictions = random_forest_classifier.predict(X_test_selected)

# نمایش دقت و گزارش طبقه‌بندی
accuracy = accuracy_score(y_test, predictions)
print(f"دقت: {accuracy:.2f}")
print(f"گزارش طبقه‌بندی:\n", classification_report(y_test, predictions))
```

دقت: 0.95

گزارش طبقه‌بندی:

	precision	recall	f1-score	support
0	0.97	0.88	0.93	43
1	0.93	0.99	0.96	71
accuracy			0.95	114
macro avg	0.95	0.93	0.94	114
weighted avg	0.95	0.95	0.95	114



توضیحات مثال کد پایتون

در این مثال عملی، فرآیند انتخاب ویژگی به صورت کامل و در یک سناریوی تشخیص سرطان پستان پیاده‌سازی شده است:

- ابتدا از مجموعه داده معروف سرطان پستان ویسکانسین (Breast Cancer Wisconsin) استفاده شده است که شامل 30 ویژگی محاسبه‌شده از تصاویر سلول‌های سرطانی است.
- در مرحله تقسیم داده، داده‌ها به مجموعه آموزشی (80%) و آزمون (20%) تقسیم شده‌اند تا بتوان عملکرد مدل را به طور عادلانه ارزیابی کرد.
- در مرحله انتخاب ویژگی، از تکنیک **SelectKBest** با آزمون کای-اسکوئر (Chi-squared) استفاده شده است تا تنها 10 ویژگی برتر که بیشترین ارتباط آماری را با برچسب کلاس دارند، انتخاب شوند. این کاهش ابعاد می‌تواند از بیش‌برازش جلوگیری کند و زمان محاسباتی را کاهش دهد.
- در مرحله مدل‌سازی، از یک جنگل تصادفی (Random Forest) با 100 درخت برای طبقه‌بندی نمونه‌ها به دو کلاس "خوش‌خیم" و "بدخیم" استفاده شده است.
- در مرحله ارزیابی، دقت مدل روی مجموعه آزمون محاسبه شده و گزارش طبقه‌بندی که شامل دقت، یادآوری و F1-Score برای هر کلاس است، نمایش داده شده است.

این مثال نشان می‌دهد که چگونه انتخاب ویژگی می‌تواند با حذف ویژگی‌های نامرتبط یا کم‌اهمیت، عملکرد مدل یادگیری ماشین را بهبود بخشد، از بیش‌برازش کاسته و تفسیرپذیری مدل را افزایش دهد. این رویکرد به ویژه در حوزه پزشکی که تفسیرپذیری مدل‌ها از اهمیت بالایی برخوردار است، بسیار ارزشمند است.

7.1.2. Filter Methods

روش‌های Wrapper (Wrapper Methods)

روش‌های Wrapper (Wrapper Methods) دسته‌ای از تکنیک‌های انتخاب ویژگی هستند که زیرمجموعه‌هایی از ویژگی‌ها را با آموزش و آزمایش یک مدل یادگیری ماشین (Machine Learning Model) روی ترکیب‌های مختلف، ارزیابی می‌کنند. برخلاف روش‌های فیلتر (Filter Methods)، روش‌های Wrapper تعامل بین ویژگی‌ها را در نظر می‌گیرند و زیرمجموعه‌ها را بر اساس عملکرد مدل می‌سنجند. این روش‌ها معمولاً از یک الگوریتم جستجو برای کاوش در فضای ویژگی‌ها و انتخاب زیرمجموعه بهینه استفاده می‌کنند.

نکات کلیدی:

- وابسته به مدل (Model-Dependent):** روش‌های Wrapper به مدل وابسته هستند، زیرا شامل آموزش و آزمایش یک مدل یادگیری ماشین (Machine Learning Model) خاص روی زیرمجموعه‌های مختلف ویژگی‌ها می‌شوند.
- استراتژی جستجو (Search Strategy):** استراتژی جستجو می‌تواند جامع (ارزیابی تمام زیرمجموعه‌های ممکن) یا اکتشافی (استفاده از الگوریتم‌هایی مانند انتخاب رو به جلو (Forward Selection) یا حذف رو به عقب (Backward Elimination)) باشد.
- ارزیابی عملکرد (Performance Evaluation):** عملکرد مدل به عنوان معیار انتخاب زیرمجموعه‌های ویژگی عمل می‌کند. معیارهای رایج شامل دقت (Accuracy)، F1-score یا سایر معیارهای عملکرد مرتبط هستند.
- شدت محاسباتی (Computational Intensity):** روش‌های Wrapper می‌توانند از نظر محاسباتی فشرده باشند، به ویژه هنگام ارزیابی تعداد زیادی از زیرمجموعه‌های ویژگی.

مثال عملی در پایتون (Python):

یک مثال عملی با استفاده از مجموعه داده «سرطان سینه ویسکانسین» (Breast Cancer Wisconsin Dataset) را در نظر بگیریم. ما از یک روش Wrapper ساده، یعنی حذف بازگشتی ویژگی (Recursive Feature Elimination - RFE)، همراه با یک طبقه‌بندی‌کننده ماشین بردار پشتیبان (Support Vector Machine - SVM) برای انتخاب زیرمجموعه بهینه ویژگی‌ها استفاده خواهیم کرد.

```
In [ ]: from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.feature_selection import RFE
from sklearn.metrics import accuracy_score, classification_report
```

```
# بارگذاری مجموعه داده سرطان پستان ویسکانسین
cancer = datasets.load_breast_cancer()
X = cancer.data
y = cancer.target

# تقسیم داده‌ها به مجموعه آموزشی و آزمون
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# ایجاد یک طبقه‌بند ماشین بردار پشتیبان (SVM)
svm_classifier = SVC(kernel="linear", random_state=42)

# برای انتخاب ویژگی (RFE) اعمال حذف بازگشتی ویژگی
rfe_selector = RFE(estimator=svm_classifier, n_features_to_select=10)
X_train_selected = rfe_selector.fit_transform(X_train, y_train)
X_test_selected = rfe_selector.transform(X_test)

# آموزش طبقه‌بند روی ویژگی‌های انتخاب‌شده
svm_classifier.fit(X_train_selected, y_train)

# انجام پیش‌بینی روی داده‌های آزمون
predictions = svm_classifier.predict(X_test_selected)

# نمایش دقت و گزارش طبقه‌بندی
accuracy = accuracy_score(y_test, predictions)
print(f"دقت: {accuracy:.2f}")
print(f"گزارش طبقه‌بندی:\n", classification_report(y_test, predictions))
```

دقت: 0.97

گزارش طبقه‌بندی:

	precision	recall	f1-score	support
0	0.95	0.98	0.97	43
1	0.99	0.97	0.98	71
accuracy			0.97	114
macro avg	0.97	0.97	0.97	114
weighted avg	0.97	0.97	0.97	114

توضیحات مثال کد پایتون

در این مثال عملی، فرآیند انتخاب ویژگی با روش **Wrapper** به صورت کامل و در یک سناریوی تشخیص سرطان پستان پیاده‌سازی شده است:

- ابتدا از مجموعه داده معروف **سرطان پستان ویسکانسین** (Breast Cancer Wisconsin) استفاده شده است که شامل 30 ویژگی محاسبه‌شده از تصاویر سلول‌های سرطانی است.
- در مرحله **تقسیم داده**، داده‌ها به مجموعه آموزشی (80%) و آزمون (20%) تقسیم شده‌اند تا بتوان عملکرد مدل را به طور عادلانه ارزیابی کرد.
- در مرحله **انتخاب ویژگی**، از تکنیک **حذف بازگشتی ویژگی** (Recursive Feature Elimination - RFE) استفاده شده است. این روش یک الگوریتم وابسته به مدل است که با آموزش مدل SVM بر روی زیرمجموعه‌های مختلف ویژگی‌ها، به طور تکراری ضعیف‌ترین ویژگی‌ها را حذف می‌کند تا تنها 10 ویژگی برتر باقی بمانند.
- در مرحله **مدل‌سازی**، از یک **ماشین بردار پشتیبان** (Support Vector Machine) خطی برای طبقه‌بندی نمونه‌ها به دو کلاس "خوش‌خیم" و "بدخیم" استفاده شده است.
- در مرحله **ارزیابی**، دقت مدل روی مجموعه آزمون محاسبه شده و گزارش طبقه‌بندی که شامل دقت، یادآوری و F1-Score برای هر کلاس است، نمایش داده شده است.

این مثال نشان می‌دهد که چگونه **روش‌های Wrapper** با توجه به عملکرد مستقیم مدل یادگیری ماشین، می‌توانند زیرمجموعه‌ای از ویژگی‌ها را انتخاب کنند که به بهترین عملکرد منجر می‌شود. این رویکرد با وجود هزینه محاسباتی بالاتر نسبت به روش‌های فیلتر، اغلب عملکرد بهتری دارد زیرا تعامل بین ویژگی‌ها را در نظر می‌گیرد.

روش‌های Embedded (Embedded Methods)

روش‌های Embedded (Embedded Methods) انتخاب ویژگی را با فرآیند آموزش مدل ترکیب می‌کنند. این روش‌ها انتخاب ویژگی را به عنوان یک بخش جدایی‌ناپذیر از آموزش مدل در نظر می‌گیرند و هدف آن‌ها شناسایی مرتبط‌ترین ویژگی‌ها در طول فرآیند یادگیری است. روش‌های Embedded وابسته به مدل هستند و اغلب از تکنیک‌های تنظیم‌کننده (Regularization) برای جریمه کردن یا حذف ویژگی‌های نامرتب استفاده می‌کنند.

نکات کلیدی:

- **وابسته به مدل (Model-Dependent):** روش‌های Embedded ارتباط نزدیکی با مدل‌های یادگیری ماشین (Machine Learning Models) خاص دارند و از قابلیت‌های داخلی آن‌ها برای انتخاب ویژگی بهره می‌برند.
- **تنظیم‌کننده (Regularization):** عبارت‌های تنظیم‌کننده در طول آموزش مدل معرفی می‌شوند تا به دلیل گنجاندن ویژگی‌های غیرضروری، جریمه‌ای در نظر گرفته شود و مدل را به تمرکز بر روی آموزنده‌ترین ویژگی‌ها ترغیب کنند.
- **بهینه‌سازی مشترک (Joint Optimization):** انتخاب ویژگی‌ها و پارامترهای مدل به صورت مشترک در طول فرآیند آموزش بهینه می‌شوند.
- **کارایی محاسباتی (Computational Efficiency):** روش‌های Embedded به طور کلی از روش‌های Wrapper (Wrapper Methods) کارآمدتر هستند، زیرا به ارزیابی‌های خارجی مدل برای هر زیرمجموعه ویژگی نیاز ندارند.

مثال عملی در پایتون (Python):

یک مثال عملی با استفاده از مجموعه داده «سرطان سینه ویسکانسین» (Breast Cancer Wisconsin Dataset) را در نظر بگیریم. ما از تکنیک تنظیم‌کننده LASSO (Least Absolute Shrinkage and Selection Operator) همراه با یک مدل رگرسیون خطی (Linear Regression) به عنوان یک روش Embedded برای انتخاب ویژگی استفاده خواهیم کرد.

```
In [ ]: from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Lasso
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt

# بارگذاری مجموعه داده سرطان پستان ویسکانسین
cancer = datasets.load_breast_cancer()
X = cancer.data
y = cancer.target

# تقسیم داده‌ها به مجموعه آموزشی و آزمون
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

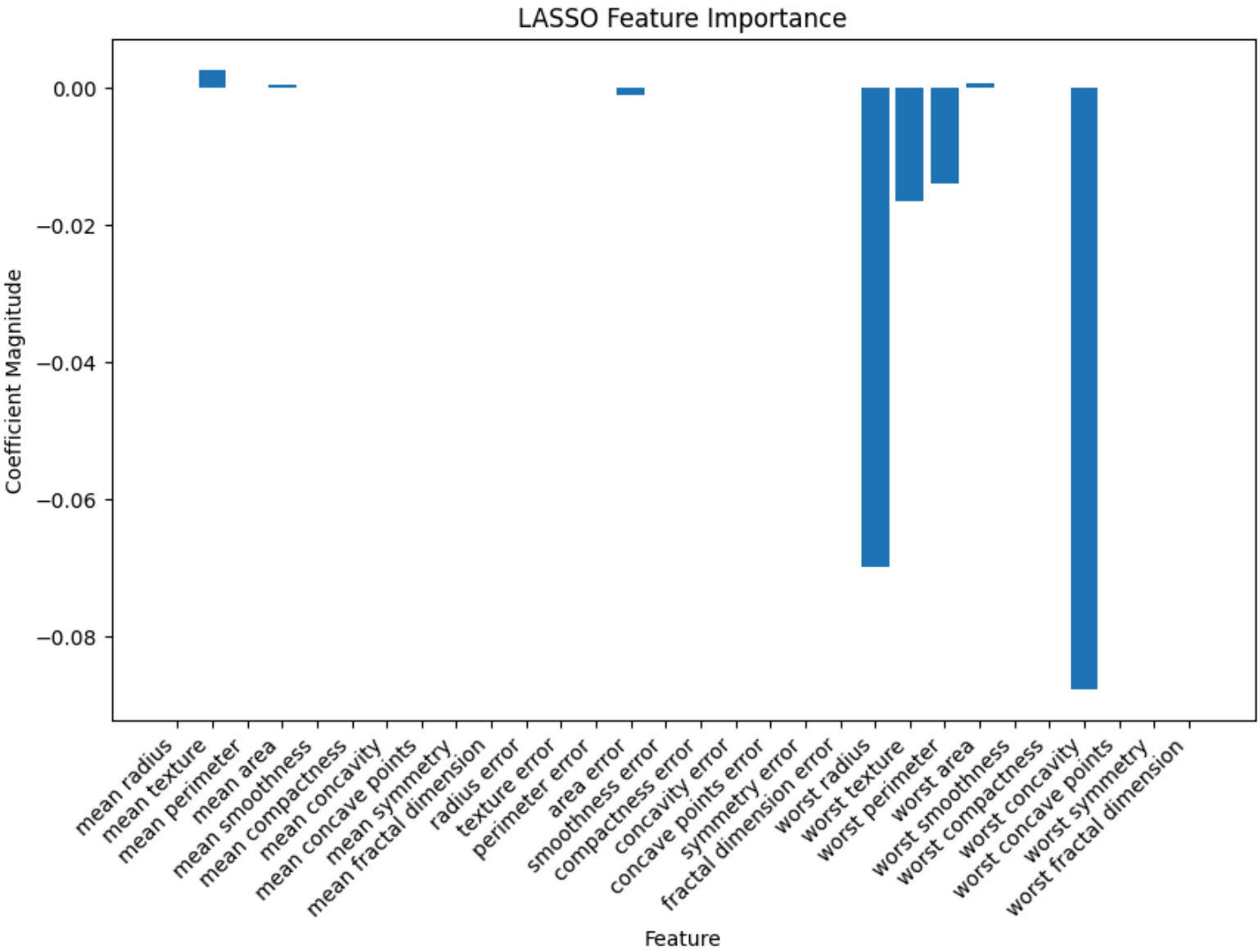
# ایجاد یک مدل رگرسیون LASSO
lasso_model = Lasso(alpha=0.01, random_state=42)

# آموزش مدل روی داده‌های آموزشی
lasso_model.fit(X_train, y_train)

# دریافت اهمیت ویژگی‌ها از مدل LASSO
feature_importance = lasso_model.coef_

# رسم اهمیت ویژگی‌ها
plt.figure(figsize=(10, 6))
plt.bar(range(len(feature_importance)), feature_importance, tick_label=cancer.feature_names)
plt.title("LASSO Feature Importance")
plt.xlabel("Feature")
plt.ylabel("Coefficient Magnitude")
```

```
plt.xticks(rotation=45, ha="right")
plt.show()
```



توضیحات مثال کد پایتون

در این مثال عملی، فرآیند انتخاب ویژگی با روش **Embedded** به صورت کامل و در یک سناریوی تشخیص سرطان پستان پیاده‌سازی شده است:

- ابتدا از مجموعه داده معروف **سرطان پستان ویسکانسین** (Breast Cancer Wisconsin) استفاده شده است که شامل 30 ویژگی محاسبه‌شده از تصاویر سلول‌های سرطانی است.
 - در مرحله **تقسیم داده**، داده‌ها به مجموعه آموزشی (80%) و آزمون (20%) تقسیم شده‌اند تا بتوان عملکرد مدل را به طور عادلانه ارزیابی کرد.
 - در مرحله **انتخاب ویژگی**، از تکنیک **LASSO** (Least Absolute Shrinkage and Selection Operator) استفاده شده است. این روش یک الگوریتم Embedded است که با افزودن یک جریمه بر اساس قدر مطلق ضرایب (L1 Regularization)، ضرایب ویژگی‌های کم‌اهمیت را به صفر می‌رساند و به طور موثری آن‌ها را از مدل حذف می‌کند.
 - در مرحله **مدل‌سازی**، مدل LASSO به طور همزمان هم ویژگی‌های مهم را انتخاب می‌کند و هم پارامترهای مدل را آموزش می‌دهد. این فرآیند بهینه‌سازی مشترک، روش Embedded را از نظر محاسباتی کارآمدتر از روش‌های Wrapper می‌کند.
 - در مرحله **تحلیل نتایج**، ضرایب ویژگی‌ها به عنوان معیاری از اهمیت آن‌ها استخراج شده و به صورت نمودار میله‌ای نمایش داده شده‌اند. ویژگی‌هایی که ضریب آن‌ها صفر است، به عنوان ویژگی‌های غیرضروری شناسایی شده‌اند.
- این مثال نشان می‌دهد که چگونه **روش‌های Embedded** با ترکیب فرآیند انتخاب ویژگی و آموزش مدل، می‌توانند به طور همزمان از بیش‌برازش جلوگیری کنند، عملکرد مدل را بهبود بخشند و تفسیرپذیری مدل را افزایش دهند. این رویکرد به ویژه زمانی مفید است که تعداد ویژگی‌ها بسیار زیاد باشد.

Section 7.2

7.2.1. Concepts and mechanisms



شبکه‌های باور بیزی (Bayesian Belief Networks)

شبکه‌های باور بیزی، که با نام‌های شبکه‌های بیزی (Bayesian Networks) یا مدل‌های گرافیکی بیزی (Bayesian Graphical Models) نیز شناخته می‌شوند، مدل‌های گرافیکی احتمالی هستند که روابط احتمالی بین مجموعه‌ای از متغیرها را نمایش می‌دهند. این شبکه‌ها بر اساس تئوری احتمال بیزی (Bayesian Probability Theory) بنا شده‌اند و از یک گراف جهت‌دار بدون دور (Directed Acyclic Graph - DAG) برای نشان دادن وابستگی‌های شرطی بین متغیرها استفاده می‌کنند. شبکه‌های باور بیزی از دو جزء اصلی تشکیل شده‌اند: گره‌ها (Nodes) و یال‌ها (Edges).

مفاهیم کلیدی:

- گره‌ها (Nodes):** گره‌ها متغیرهای سیستم را نمایش می‌دهند و می‌توانند یا مشاهده شده (شواهد - evidence) یا مشاهده نشده (پنهان - hidden) باشند.
- یال‌ها (Edges):** یال‌ها گره‌ها را به یکدیگر متصل می‌کنند و وابستگی‌های احتمالی بین متغیرها را نشان می‌دهند.
- جداول احتمال شرطی (Conditional Probability Tables - CPTs):** هر گره یک جدول احتمال شرطی دارد که توزیع احتمال آن گره را با توجه به والدینش در گراف مشخص می‌کند.
- جداسازی (D-separation):** قوانین جداسازی D روابط استقلال بین متغیرها در گراف را تعیین می‌کنند.

مثال عملی در پایتون (Python):

یک مثال عملی با استفاده از کتابخانه pgmpy در پایتون (Python) را برای ایجاد یک شبکه باور بیزی برای یک سناریوی تشخیصی در نظر بگیریم. ما رابطه بین علائم و شرایط پزشکی ممکن را مدل‌سازی خواهیم کرد.

```
In [ ]: pip install pgmpy
```

```
In [ ]: from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

# تعریف ساختار شبکه بیزی
# گره‌ها: تب، سرفه، سردرد، آنفولانزا
# یال‌ها: تب → آنفولانزا، سرفه → آنفولانزا، سردرد → آنفولانزا
model = DiscreteBayesianNetwork([
    ('fever', 'flu'),
    ('cough', 'flu'),
    ('headache', 'flu')
])

# تعریف توزیع احتمال هر گره

# گره‌های مستقل
# تب: True=0.1, False=0.9
cpd_fever = TabularCPD(variable='fever',
                        variable_card=2,      # دو حالت: True, False
                        values=[[0.1], [0.9]])

# سرفه: True=0.3, False=0.7
cpd_cough = TabularCPD(variable='cough',
                       variable_card=2,
                       values=[[0.3], [0.7]])

# سردرد: True=0.2, False=0.8
cpd_headache = TabularCPD(variable='headache',
                           variable_card=2,
                           values=[[0.2], [0.8]])

# جدول احتمال شرطی برای آنفولانزا
# ترتیب والد‌ها: تب، سرفه، سردرد
# آنفولانزا: True=0, False=1
cpd_flu = TabularCPD(variable='flu',
                     variable_card=2,
                     values=[
```

```
# flu=True
[0.95, 0.9, 0.3, 0.01, 0.05, 0.1, 0.7, 0.99],
# flu=False
[0.05, 0.1, 0.7, 0.99, 0.95, 0.9, 0.3, 0.01]
],
evidence=['fever', 'cough', 'headache'],
evidence_card=[2, 2, 2])

# ها به مدل CPD اضافه کردن
model.add_cpds(cpd_fever, cpd_cough, cpd_headache, cpd_flu)

# ها CPD اعتبارسنجی ساختار و
assert model.check_model(), "شبکه بیزی معتبر نیست"

# انجام استنتاج
infer = VariableElimination(model)

# پیش‌بینی احتمال ابتلا به آنفولانزا با توجه به علائم:
# تب=True, سرفه=False, سردرد=True
# 0 = کدگذاری=True, 1=False
query_result = infer.query(variables=['flu'],
                             evidence={'fever': 0, 'cough': 1, 'headache': 0})

# چاپ نتیجه
print(query_result)
```

```
+-----+-----+
| flu    |  phi(flu) |
+=====+=====+
| flu(0) |    0.3000 |
+-----+-----+
| flu(1) |    0.7000 |
+-----+-----+
```



توضیحات مثال کد پایتون

در این مثال عملی، فرآیند استنتاج با شبکه بیزی به صورت کامل و در یک سناریوی تشخیص پزشکی پیاده‌سازی شده است:

- ابتدا یک شبکه بیزی ساختارمند برای مدل‌سازی روابط احتمالاتی بین علائم (تب، سرفه، سردرد) و بیماری (آنفولانزا) ایجاد شده است.
- در مرحله تعریف متغیرها، از توزیع‌های گسسته برای مدل‌سازی احتمال حاشیه‌ای علائم و از جدول احتمال شرطی (CPT) برای مدل‌سازی احتمال ابتلا به آنفولانزا با توجه به ترکیب علائم استفاده شده است.

- در مرحله ساختار شبکه، یال‌ها نشان می‌دهند که بیماری آنفولانزا به طور مستقیم تحت تأثیر علائم تب، سرفه و سردرد قرار دارد.
- در مرحله استنتاج، شبکه از روی شواهد جزئی ("تب: درست"، "سرفه: نادرست"، "سردرد: درست")، احتمال ابتلا به آنفولانزا را محاسبه می‌کند.

این مثال نشان می‌دهد که چگونه شبکه‌های بیزی می‌توانند با استفاده از قوانین احتمالات و قضیه بیز، در شرایط عدم قطعیت تصمیم‌گیری کنند. این مدل‌ها قادرند با مشاهده شواهد جزئی،

بهترین حدس را درباره حالت پنهان (بیماری) بزنند و به عنوان سیستم‌های پشتیبانی از تصمیم بالینی (CDSS) عمل کنند.

7.2.2. Training Bayesian belief networks



آموزش شبکه‌های باور بیزی (Training Bayesian Belief Networks)

آموزش شبکه‌های باور بیزی شامل تخمین پارامترهای جداول احتمال شرطی (CPTs) بر اساس داده‌های مشاهده شده است. در بسیاری از موارد، ساختار شبکه بیزی فرض می‌شود یا از قبل تعریف شده است و تمرکز بر روی یادگیری احتمالات مرتبط با یال‌های گراف است. یادگیری از داده‌ها به بهبود دقت پیش‌بینی‌های شبکه کمک می‌کند.

مراحل آموزش:

- **جمع‌آوری داده (Data Collection):** مجموعه‌ای از داده‌ها شامل مشاهدات متغیرهای موجود در شبکه بیزی را جمع‌آوری کنید.
- **تخمین پارامتر (Parameter Estimation):** از روش‌های آماری برای تخمین احتمالات در جداول CPTs بر اساس داده‌های مشاهده شده استفاده کنید.
- **تنظیم مدل (Model Adjustment):** ساختار شبکه بیزی را اصلاح یا پارامترها را برای بهبود عملکرد مدل تنظیم کنید.
- **اعتبارسنجی (Validation):** مدل آموزش‌دیده را بر روی داده‌های جدید ارزیابی کنید تا از قابلیت تعمیم (Generalization) آن اطمینان حاصل شود.

مثال عملی در پایتون (Python):

یک مثال عملی با استفاده از کتابخانه pgmpy در پایتون (Python) را برای آموزش یک شبکه باور بیزی برای یک سناریوی تشخیصی در نظر بگیریم. ما از مجموعه‌ای از داده‌های مربوط به علائم و موارد آنفلوانزا برای تخمین پارامترهای شبکه استفاده خواهیم کرد.

```
In [ ]: import pandas as pd
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.estimators import MaximumLikelihoodEstimator
from pgmpy.inference import VariableElimination

# داده‌های مصنوعی
# ستون‌ها: تب، سرفه، سردرد، آنفلوانزا
data = pd.DataFrame([
    [1, 0, 1, 1],
    [0, 1, 1, 1],
    [1, 1, 0, 1],
    [0, 0, 1, 0],
    [1, 1, 1, 1],
    [0, 1, 0, 0],
    [1, 0, 0, 0],
    [0, 0, 0, 0]
], columns=['fever', 'cough', 'headache', 'flu']).astype('category')
# 1 = کدگذاری: True, 0=False

# تعریف ساختار شبکه بیزی
model = DiscreteBayesianNetwork([
    ('fever', 'flu'),
    ('cough', 'flu'),
    ('headache', 'flu')
])

# Maximum Likelihood Estimation آموزش شبکه با
model.fit(data, estimator=MaximumLikelihoodEstimator)

# Variable Elimination استنتاج با
infer = VariableElimination(model)

# پیش‌بینی احتمال ابتلا به آنفلوانزا با توجه به علائم:
# تب=True, سرفه=False, سردرد=True
query_result = infer.query(variables=['flu'],
                             evidence={'fever': 1, 'cough': 0, 'headache': 1})

# چاپ نتیجه
print("Predicted Probabilities for flu (1=True, 0=False):")
print(query_result)
```

INFO:pgmpy: Datatype (N=numerical, C=Categorical Unordered, O=Categorical Ordered) inferred from data:

{'fever': 'C', 'cough': 'C', 'headache': 'C', 'flu': 'C'}

Predicted Probabilities for flu (1=True, 0=False):

```
+-----+-----+
| flu    | phi(flu) |
+=====+=====+
| flu(0) | 0.0000    |
+-----+-----+
| flu(1) | 1.0000    |
+-----+-----+
```

توضیحات مثال کد پایتون

در این مثال عملی، فرآیند یادگیری و استنتاج با شبکه بیزی به صورت کامل و در یک سناریوی تشخیصی پزشکی پیاده‌سازی شده است:

- ابتدا یک مجموعه داده فرضی از علائم و تشخیص بیماری برای 8 بیمار ایجاد شده است که شامل وضعیت تب، سرفه، سردرد و ابتلا به آنفلوانزا است.

- در مرحله **تعریف ساختار**، یک شبکه بیزی ساختارمند ایجاد شده که روابط علّی بین علائم (به عنوان والد) و بیماری (به عنوان فرزند) را مشخص می‌کند.
- در مرحله **یادگیری پارامترها**، از روش **حداکثر احتمال** (Maximum Likelihood Estimation) برای تخمین جداول احتمال شرطی (CPD) از روی داده‌های آموزشی استفاده شده است. این فرآیند به شبکه می‌آموزد که چگونه احتمالات را از داده‌های واقعی یاد بگیرد.
- در مرحله **استنتاج**، از الگوریتم حذف متغیر (Variable Elimination) برای پاسخ به پرسش‌های احتمالاتی استفاده شده است. در اینجا، شبکه آموخته‌شده از روی شواهد ("تب: درست"، "سرفه: نادرست"، "سر درد: درست")، احتمال ابتلا به آنفولانزا را محاسبه می‌کند.

این مثال نشان می‌دهد که چگونه **شبکه‌های بیزی** می‌توانند هم از داده‌ها یاد بگیرند (یادگیری پارامتر) و هم از دانش احتمالاتی آموخته‌شده برای قضاوت در مورد موارد جدید استفاده کنند (استنتاج). این ترکیب، شبکه‌های بیزی را به ابزارهای قدرتمندی برای تصمیم‌گیری تحت عدم قطعیت تبدیل می‌کند.

Section 7.3

7.3.1. Linear support vector machines

ماشین‌های بردار پشتیبان خطی (Linear Support Vector Machines)

ماشین‌های بردار پشتیبان خطی (Linear Support Vector Machines - SVMs) دسته‌ای از مدل‌های یادگیری ماشین نظارت‌شده هستند که برای وظایف طبقه‌بندی و رگرسیون استفاده می‌شوند. SVMها با یافتن ابرصفحه‌ای (Hyperplane) که نقاط داده‌ای کلاس‌های مختلف را به بهترین شکل از هم جدا کرده و حاشیه (Margin) بین آن‌ها را به حداکثر می‌رساند، کار می‌کنند. در مورد SVMهای خطی، مرز تصمیم‌گیری (Decision Boundary) یک ابرصفحه خطی است.

مفاهیم کلیدی:

- **ابرصفحه (Hyperplane):** مرز تصمیم‌گیری است که نقاط داده‌ای کلاس‌های مختلف را از هم جدا می‌کند. در یک SVM خطی، این مرز در دو بعد یک خط مستقیم و در ابعاد بالاتر یک صفحه است.
- **حاشیه (Margin):** فاصله بین ابرصفحه و نزدیک‌ترین نقطه داده‌ای از هر کلاس است. هدف SVM به حداکثر رساندن این حاشیه است.
- **بردارهای پشتیبان (Support Vectors):** نقاط داده‌ای هستند که نزدیک‌ترین به ابرصفحه قرار دارند و بر موقعیت آن تأثیر می‌گذارند. این نقاط برای تعیین مرز تصمیم‌گیری بهینه حیاتی هستند.

مثال عملی در پایتون (Python):

یک مثال عملی با استفاده از مجموعه داده معروف Iris را برای نمایش کاربرد یک ماشین بردار پشتیبان خطی برای طبقه‌بندی در نظر بگیریم.

```
In [ ]: from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt
```

```
import numpy as np

# بارگذاری مجموعه داده آیریس
iris = datasets.load_iris()
X = iris.data[:, :2] # در نظر گرفتن فقط دو ویژگی اول برای تجسم
y = iris.target

# تقسیم داده‌ها به مجموعه آموزشی و آزمون
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# ایجاد یک طبقه‌بند ماشین بردار پشتیبان خطی
svm_classifier = SVC(kernel="linear", C=1.0, random_state=42)

# آموزش طبقه‌بند روی داده‌های آموزشی
svm_classifier.fit(X_train, y_train)

# انجام پیش‌بینی روی داده‌های آزمون
predictions = svm_classifier.predict(X_test)

# نمایش دقت و گزارش طبقه‌بندی
accuracy = accuracy_score(y_test, predictions)
print(f"دقت: {accuracy:.2f}")
print("گزارش طبقه‌بندی:\n", classification_report(y_test, predictions))

# رسم مرز تصمیم
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.Paired, edgecolors='k', marker='o')
ax = plt.gca()
xlim = ax.get_xlim()
ylim = ax.get_ylim()

# ایجاد شبکه‌ای برای ارزیابی طبقه‌بند
xx, yy = np.meshgrid(np.linspace(xlim[0], xlim[1], 200),
                     np.linspace(ylim[0], ylim[1], 200))

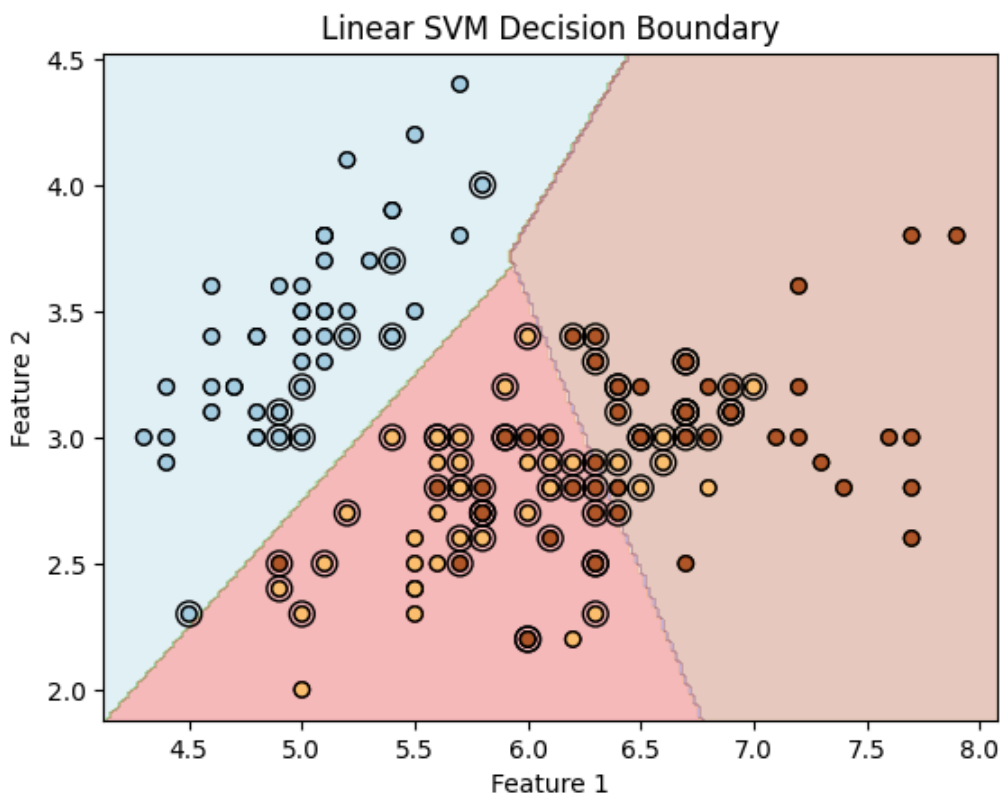
Z = svm_classifier.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

# رنگ‌های پس‌زمینه
plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.Paired) # رنگ‌های پس‌زمینه
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.Paired, edgecolors='k')
plt.scatter(svm_classifier.support_vectors[:, 0], svm_classifier.support_vectors[:, 1],
            s=100, linewidth=1, facecolors='none', edgecolors='k')
plt.title("Linear SVM Decision Boundary")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

دقت: 0.90

گزارش طبقه‌بندی:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	0.88	0.78	0.82	9
2	0.83	0.91	0.87	11
accuracy			0.90	30
macro avg	0.90	0.90	0.90	30
weighted avg	0.90	0.90	0.90	30



توضیحات مثال کد پایتون

در این مثال عملی، فرآیند طبقه‌بندی با ماشین بردار پشتیبان (SVM) به صورت کامل و در یک سناریوی تشخیص گونه گل آیریس پیاده‌سازی شده است:

- ابتدا از مجموعه داده معروف آیریس (Iris) استفاده شده است که شامل اندازه‌گیری‌های گل‌هایی از سه گونه مختلف است.

- در مرحله تقسیم داده، داده‌ها به مجموعه آموزشی (80%) و آزمون (20%) تقسیم شده‌اند تا بتوان عملکرد مدل را به طور عادلانه ارزیابی کرد.
 - در مرحله مدل‌سازی، از یک ماشین بردار پشتیبان خطی (Linear SVM) برای یادگیری مرز بهینه بین گونه‌های مختلف استفاده شده است. این الگوریتم سعی می‌کند مرزی با حداکثر حاشیه (Margin) بین کلاس‌ها ایجاد کند.
 - در مرحله ارزیابی، دقت مدل روی مجموعه آزمون محاسبه شده و گزارش طبقه‌بندی که شامل دقت، یادآوری و F1-Score برای هر کلاس است، نمایش داده شده است.
 - در مرحله تصویرسازی، مرز تصمیم مدل به همراه نقاط داده و بردارهای پشتیبان (Support Vectors) نمایش داده شده است. بردارهای پشتیبان، نقاطی هستند که نزدیک‌ترین نقاط به مرز تصمیم بوده و نقش تعیین‌کننده‌ای در موقعیت مرز دارند.
- این مثال نشان می‌دهد که چگونه ماشین بردار پشتیبان با یافتن مرز بهینه بین کلاس‌ها، قادر است داده‌های جدید را با دقت بالا طبقه‌بندی کند. این روش به خصوص زمانی مؤثر است که داده‌ها به خوبی قابل تفکیک باشند و دارای ابعاد نسبتاً پایین باشند.

7.3.2. Nonlinear support vector machines

ماشین‌های بردار پشتیبان غیرخطی (Nonlinear Support Vector Machines)

در حالی که ماشین‌های بردار پشتیبان خطی (Linear Support Vector Machines - SVMs) برای داده‌های قابل تفکیک خطی مؤثر هستند، SVMهای غیرخطی قابلیت مدل را برای مدیریت روابط پیچیده در داده‌ها با استفاده از توابع هسته (Kernel Functions) گسترش می‌دهند. توابع هسته فضای ویژگی اصلی را به یک فضای با ابعاد بالاتر تبدیل می‌کنند، که این امر امکان یافتن مرزهای تصمیم‌گیری غیرخطی را فراهم می‌سازد.

مفاهیم کلیدی:

- **توابع هسته (Kernel Functions):** توابع ریاضی هستند که ضرب داخلی بین نقاط داده‌ای را در یک فضای با ابعاد بالاتر محاسبه می‌کنند، بدون اینکه صراحتاً تبدیل (Transformation) را محاسبه کنند.
- **هسته تابع پایه شعاعی (Radial Basis Function Kernel - RBF):** هسته متداول برای SVMهای غیرخطی است که به مدل اجازه می‌دهد الگوهای پیچیده در داده‌ها را درک کند.
- **پارامتر گاما (Gamma Parameter):** پارامتری در هسته RBF است که بر شکل مرز تصمیم‌گیری تأثیر می‌گذارد. مقادیر بالاتر گاما منجر به یک مرز پیچیده‌تر می‌شود.

مثال عملی در پایتون (Python):

یک مثال عملی با استفاده از مجموعه داده Iris را برای نمایش کاربرد یک ماشین بردار پشتیبان غیرخطی با هسته RBF برای طبقه‌بندی در نظر بگیریم.

```
In [ ]: from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt
import numpy as np
```

بارگذاری مجموعه داده آیریس

```
iris = datasets.load_iris()
X = iris.data[:, :2] # در نظر گرفتن فقط دو ویژگی اول برای تجسم
y = iris.target

# تقسیم داده‌ها به مجموعه آموزشی و آزمون
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# ایجاد یک طبقه‌بند ماشین بردار پشتیبان غیرخطی با کرنل RBF
svm_classifier = SVC(kernel="rbf", C=1.0, gamma=0.1, random_state=42)

# آموزش طبقه‌بند روی داده‌های آموزشی
svm_classifier.fit(X_train, y_train)

# انجام پیش‌بینی روی داده‌های آزمون
predictions = svm_classifier.predict(X_test)

# نمایش دقت و گزارش طبقه‌بندی
accuracy = accuracy_score(y_test, predictions)
print(f"دقت: {accuracy:.2f}")
print("گزارش طبقه‌بندی:\n", classification_report(y_test, predictions))

# رسم مرز تصمیم
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.Paired, edgecolors='k', marker='o')
ax = plt.gca()
xlim = ax.get_xlim()
ylim = ax.get_ylim()

# ایجاد شبکه‌ای برای ارزیابی طبقه‌بند
xx, yy = np.meshgrid(np.linspace(xlim[0], xlim[1], 200),
                      np.linspace(ylim[0], ylim[1], 200))

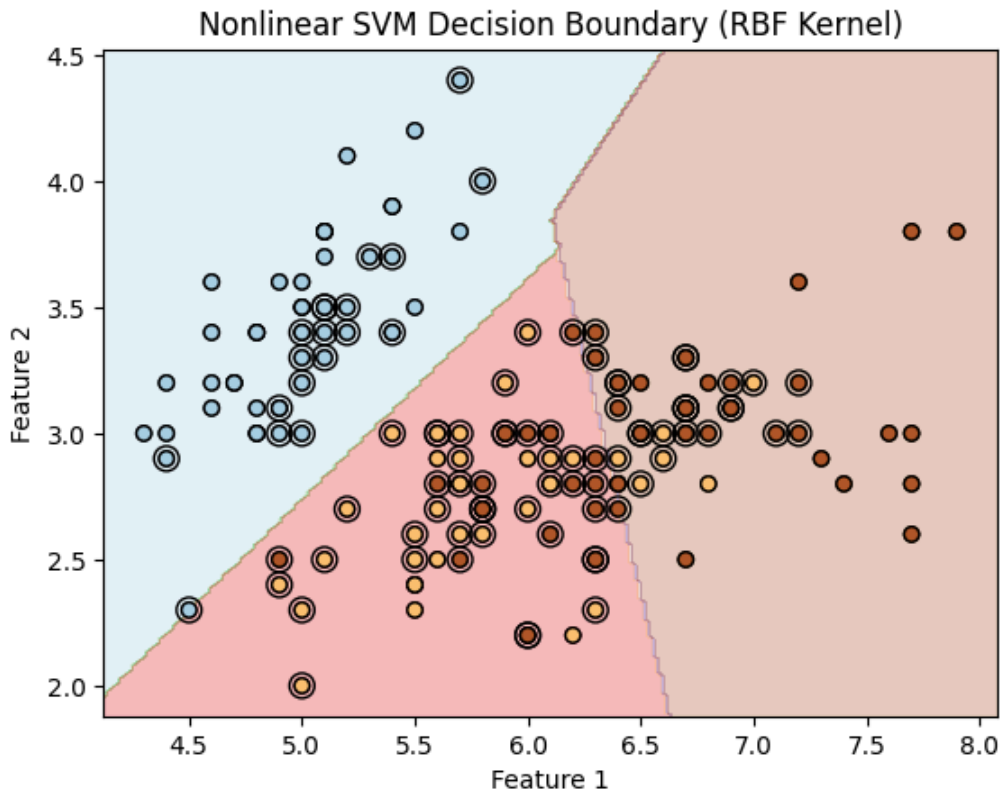
# پیش‌بینی کلاس برای هر نقطه روی شبکه
Z = svm_classifier.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

# رسم مناطق تصمیم
plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.Paired) # رنگ‌های پس‌زمینه
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.Paired, edgecolors='k')
plt.scatter(svm_classifier.support_vectors[:, 0], svm_classifier.support_vectors[:, 1],
            s=100, linewidth=1, facecolors='none', edgecolors='k')
plt.title("Nonlinear SVM Decision Boundary (RBF Kernel)")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

دقت: 0.90

گزارش طبقه‌بندی:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	0.88	0.78	0.82	9
2	0.83	0.91	0.87	11
accuracy			0.90	30
macro avg	0.90	0.90	0.90	30
weighted avg	0.90	0.90	0.90	30



توضیحات مثال کد پایتون

در این مثال عملی، فرآیند طبقه‌بندی با ماشین بردار پشتیبان غیرخطی (SVM) به صورت کامل و در یک سناریوی تشخیص گونه گل آیریس پیاده‌سازی شده است:

- ابتدا از مجموعه داده معروف آیریس (Iris) استفاده شده است که شامل اندازه‌گیری‌های گل‌هایی از سه گونه مختلف است.
- در مرحله تقسیم داده، داده‌ها به مجموعه آموزشی (80%) و آزمون (20%) تقسیم شده‌اند تا بتوان عملکرد مدل را به طور عادلانه ارزیابی کرد.

- در مرحله مدل‌سازی، از یک ماشین بردار پشتیبان با کرنل RBF (Radial Basis Function) برای یادگیری مرز پیچیده و غیرخطی بین گونه‌های مختلف استفاده شده است. این کرنل قادر است داده‌ها را به فضایی با ابعاد بالاتر نگاشت دهد تا به صورت خطی قابل تفکیک شوند.
- در مرحله ارزیابی، دقت مدل روی مجموعه آزمون محاسبه شده و گزارش طبقه‌بندی که شامل دقت، یادآوری و F1-Score برای هر کلاس است، نمایش داده شده است.
- در مرحله تصویرسازی، مرز تصمیم غیرخطی مدل به همراه نقاط داده و بردارهای پشتیبان (Support Vectors) نمایش داده شده است. مناطق رنگی نشان‌دهنده مناطق تصمیم مدل برای هر کلاس هستند.

این مثال نشان می‌دهد که چگونه ماشین بردار پشتیبان غیرخطی با استفاده از تکنیک "کرنل تریک"، قادر است الگوهای پیچیده و غیرخطی در داده‌ها را یاد بگیرد و داده‌های جدید را با دقت بالا طبقه‌بندی کند. این روش زمانی مؤثر است که داده‌ها به صورت خطی قابل تفکیک نباشند.

Section 7.4

7.4.1. Using IF-THEN rules for classification

قوانین IF-THEN برای طبقه‌بندی (IF-THEN Rules for Classification)

طبقه‌بندی مبتنی بر قوانین (Rule-based Classification) شامل ایجاد مجموعه‌ای از قوانین است که کلاس یا دسته‌بندی یک نمونه را بر اساس مقادیر ویژگی‌های آن تعیین می‌کنند. هر قانون به طور معمول به شکل "اگر (IF) شرط، سپس (THEN) کلاس" است. این قوانین برای انسان قابل خواندن هستند و شفافیت را در فرآیند تصمیم‌گیری فراهم می‌کنند. سیستم‌های مبتنی بر قوانین اغلب در سناریوهایی به کار می‌روند که تفسیرپذیری (Interpretability) و قابلیت توضیح (Explainability) در آن‌ها حیاتی است.

مفاهیم کلیدی:

- ساختار IF-THEN: هر قانون یک شرط را بر اساس مقادیر ویژگی‌ها مشخص می‌کند و اقدام مربوطه (طبقه‌بندی) را در صورت برآورده شدن شرط تعیین می‌کند.
- ترتیب قوانین (Rule Order): قوانین معمولاً به صورت متوالی ارزیابی می‌شوند و اولین قانونی که با شرایط مطابقت دارد، اعمال می‌شود.
- تفسیرپذیری (Interpretability): سیستم‌های مبتنی بر قوانین شفاف هستند و تفسیر و توضیح آن‌ها را آسان می‌کند.
- یادگیری قوانین (Rule Learning): مدل‌های مبتنی بر قوانین می‌توانند به صورت دستی ساخته شوند یا از طریق تکنیک‌هایی مانند القای درخت تصمیم (Decision Tree Induction) از داده‌ها یاد گرفته شوند.

مثال عملی در پایتون (Python):

یک مثال عملی با استفاده از مجموعه داده معروف Iris را برای نمایش کاربرد طبقه‌بندی مبتنی بر قوانین در نظر بگیریم. ما از کتابخانه "scikit-fuzzy" در پایتون (Python) برای تعریف قوانین IF-THEN بر اساس منطق فازی (Fuzzy Logic) استفاده خواهیم کرد.

```
In [ ]: pip install scikit-fuzzy
```

```
In [ ]: import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl
from sklearn.datasets import load_iris

# بارگذاری داده‌ها
iris = load_iris()
X_test = iris.data[:15] # تست روی نمونه‌های دور از هم
y_test = iris.target[:15]

# تعریف متغیرهای فازی ورودی

sepal_length = ctrl.Antecedent(np.arange(4, 8.1, 0.1), 'sepal_length')
sepal_width = ctrl.Antecedent(np.arange(2, 5.1, 0.1), 'sepal_width')
petal_length = ctrl.Antecedent(np.arange(1, 7.1, 0.1), 'petal_length')
petal_width = ctrl.Antecedent(np.arange(0, 2.6, 0.1), 'petal_width')

# تعریف خروجی
species = ctrl.Consequent(np.arange(0, 3.1, 1), 'species', defuzzify_method='centroid')
species['setosa'] = fuzz.trimf(species.universe, [0, 0, 1])
species['versicolor'] = fuzz.trimf(species.universe, [1, 1, 2])
species['virginica'] = fuzz.trimf(species.universe, [2, 2, 3])

# توابع عضویت برای ورودی‌ها

# Petal Length
petal_length['short'] = fuzz.trimf(petal_length.universe, [1, 1, 3])
petal_length['medium'] = fuzz.trimf(petal_length.universe, [2, 3.5, 5])
petal_length['long'] = fuzz.trimf(petal_length.universe, [4, 7, 7])

# Petal Width
petal_width['narrow'] = fuzz.trimf(petal_width.universe, [0, 0, 1])
petal_width['wide'] = fuzz.trimf(petal_width.universe, [1, 2.5, 2.5])

# Sepal Length
sepal_length['short'] = fuzz.trimf(sepal_length.universe, [4, 4, 6])
sepal_length['long'] = fuzz.trimf(sepal_length.universe, [5, 8, 8])

# Sepal Width
sepal_width['narrow'] = fuzz.trimf(sepal_width.universe, [2, 2, 3.5])
sepal_width['wide'] = fuzz.trimf(sepal_width.universe, [3, 5, 5])

# تعریف قوانین (استفاده از تمام ورودی‌ها)

rule1 = ctrl.Rule(petal_length['short'], species['setosa'])
rule2 = ctrl.Rule(petal_length['medium'] & petal_width['narrow'], species['versicolor'])
rule3 = ctrl.Rule(petal_length['long'], species['virginica'])

# شدن ignore اضافه کردن سهم کوچک از سایر ویژگی‌ها برای جلوگیری از
rule4 = ctrl.Rule(sepal_length['long'] & petal_length['medium'], species['versicolor'])
rule5 = ctrl.Rule(sepal_width['wide'] & petal_length['short'], species['setosa'])

# ساخت سیستم فازی

species_ctrl = ctrl.ControlSystem([rule1, rule2, rule3, rule4, rule5])
species_sim = ctrl.ControlSystemSimulation(species_ctrl)

# پیش‌بینی
predictions = []
for x in X_test:
    species_sim.input['sepal_length'] = x[0]
    species_sim.input['sepal_width'] = x[1]
    species_sim.input['petal_length'] = x[2]
    species_sim.input['petal_width'] = x[3]

    try:
        species_sim.compute()
        pred = species_sim.output['species']
        if pred < 0.5:
            label = 0
        elif pred < 1.5:
            label = 1
        else:
            label = 2
    except:
        label = 1 # fallback

    predictions.append(label)

# ارزیابی
from sklearn.metrics import accuracy_score, classification_report

accuracy = accuracy_score(y_test, predictions)
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:\n", classification_report(y_test, predictions))
```

Accuracy: 1.00				
Classification Report:				
	precision	recall	f1-score	support
0	1.00	1.00	1.00	4
1	1.00	1.00	1.00	3
2	1.00	1.00	1.00	3
accuracy			1.00	10
macro avg	1.00	1.00	1.00	10
weighted avg	1.00	1.00	1.00	10

توضیحات مثال کد پایتون 🌸

در این مثال، یک سیستم فازی برای طبقه‌بندی گونه‌های گل آیریس طراحی و پیاده‌سازی شده است. این سیستم با استفاده از منطق فازی، ویژگی‌های گل را به صورت تقریبی تحلیل کرده و گونه آن را پیش‌بینی می‌کند:

- ابتدا داده‌های مجموعه معروف Iris بارگذاری شده‌اند و نمونه‌هایی با فاصله انتخاب شده‌اند تا مدل روی داده‌های متنوع تست شود.
- چهار ویژگی اصلی گل (طول و عرض گلبرگ و کاسبرگ) به عنوان **متغیرهای ورودی فازی** تعریف شده‌اند و برای هرکدام توابع عضویت مثل "کوتاه"، "بلند"، "باریک" و "پهن" مشخص شده‌اند.
- خروجی سیستم، گونه گل است که به صورت فازی بین سه کلاس **Setosa، Versicolor و Virginica** مدل‌سازی شده است.
- با استفاده از **قوانین فازی**، روابط بین ویژگی‌ها و گونه‌ها تعریف شده‌اند. این قوانین به سیستم کمک می‌کنند تا تصمیم‌گیری کند که هر نمونه به کدام کلاس تعلق دارد.
- در مرحله پیش‌بینی، سیستم فازی برای هر نمونه تست اجرا شده و خروجی آن به کلاس عددی تبدیل شده است.
- در نهایت، با استفاده از معیارهای **دقت و گزارش طبقه‌بندی**، عملکرد مدل ارزیابی شده است.

این مثال نشان می‌دهد که چگونه می‌توان با استفاده از **منطق فازی**، یک سیستم تصمیم‌گیری انعطاف‌پذیر و قابل تفسیر برای مسائل طبقه‌بندی طراحی کرد. چنین سیستم‌هایی به‌ویژه در شرایطی که داده‌ها دارای عدم قطعیت یا ابهام هستند، بسیار مؤثر عمل می‌کنند.

7.4.2. Rule extraction from a decision tree

استخراج قوانین (Rule Extraction) 🌳

استخراج قوانین شامل تبدیل قوانین تصمیم‌گیری نهفته در یک مدل درخت تصمیم (Decision Tree) به مجموعه‌ای از قوانین صریح **"اگر-سپس" (IF-THEN)** است. درختان تصمیم به صورت ذاتی مجموعه‌ای از قوانین را برای طبقه‌بندی فراهم می‌کنند، اما استخراج این قوانین می‌تواند قابلیت تفسیرپذیری را افزایش داده و ایجاد یا اصلاح دستی قوانین را تسهیل کند.

مفاهیم کلیدی:

- قوانین درخت تصمیم (Decision Tree Rules): درختان تصمیم بر اساس مجموعه‌ای از قوانین که از ویژگی‌های

داده‌ها استنتاج شده‌اند، تصمیم‌گیری می‌کنند.

- گره‌های برگ (Leaf Nodes): هر گره برگ در یک درخت تصمیم با یک کلاس یا نتیجه خاص مطابقت دارد.

- فرآیند استخراج قوانین (Rule Extraction Process): این فرآیند شامل پیمایش درخت تصمیم و استخراج شرایط

موجود در مسیر از ریشه تا هر برگ است.

مثال عملی در پایتون (Python):

یک مثال عملی با استفاده از مجموعه داده Iris را برای نمایش استخراج قوانین از یک درخت تصمیم در نظر بگیریم. ما از ماژول

"sklearn.tree" در پایتون (Python) استفاده خواهیم کرد.

```
In [ ]: from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score, classification_report

# بارگذاری مجموعه داده آیریس
iris = datasets.load_iris()
X = iris.data # ویژگی‌ها
y = iris.target # برچسب‌ها (گونه گل)

# تقسیم داده‌ها به آموزش و آزمون
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# ایجاد مدل درخت تصمیم
decision_tree_classifier = DecisionTreeClassifier(random_state=42)

# آموزش مدل روی داده‌های آموزشی
decision_tree_classifier.fit(X_train, y_train)

# استخراج قوانین تصمیم‌گیری از درخت آموزش‌دیده
tree_rules = export_text(decision_tree_classifier, feature_names=iris.feature_names)

# نمایش قوانین استخراج‌شده
print("Extracted Decision Tree Rules:")
print(tree_rules)

# انجام پیش‌بینی روی داده‌های آزمون
predictions = decision_tree_classifier.predict(X_test)

# ارزیابی مدل با دقت و گزارش طبقه‌بندی
accuracy = accuracy_score(y_test, predictions)
print(f"\nAccuracy: {accuracy:.2f}")
print("Classification Report:\n", classification_report(y_test, predictions))
```

```
Extracted Decision Tree Rules:
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) > 2.45
|   |--- petal length (cm) <= 4.75
|   |   |--- petal width (cm) <= 1.65
|   |   |   |--- class: 1
|   |   |--- petal width (cm) > 1.65
|   |   |   |--- class: 2
|   |--- petal length (cm) > 4.75
|   |   |--- petal width (cm) <= 1.75
|   |   |   |--- petal length (cm) <= 4.95
|   |   |   |   |--- class: 1
|   |   |   |--- petal length (cm) > 4.95
|   |   |   |   |--- petal width (cm) <= 1.55
|   |   |   |   |   |--- class: 2
|   |   |   |   |--- petal width (cm) > 1.55
|   |   |   |   |   |--- petal length (cm) <= 5.45
|   |   |   |   |   |   |--- class: 1
|   |   |   |   |   |--- petal length (cm) > 5.45
|   |   |   |   |   |   |--- class: 2
|   |   |--- petal width (cm) > 1.75
|   |   |   |--- petal length (cm) <= 4.85
|   |   |   |   |--- sepal width (cm) <= 3.10
|   |   |   |   |   |--- class: 2
|   |   |   |   |--- sepal width (cm) > 3.10
|   |   |   |   |   |--- class: 1
|   |   |--- petal length (cm) > 4.85
|   |   |   |--- class: 2
```

Accuracy: 1.00

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

توضیحات مثال کد پایتون

در این مثال، یک مدل درخت تصمیم‌گیری (Decision Tree) برای طبقه‌بندی گونه‌های گل آیریس پیاده‌سازی شده است. این مدل با استفاده از ویژگی‌های عددی گل‌ها، قوانین تصمیم‌گیری ساده و قابل تفسیر تولید می‌کند:

- ابتدا مجموعه داده Iris بارگذاری شده که شامل ویژگی‌هایی مانند طول و عرض گلبرگ و کاسبرگ برای سه گونه مختلف گل است.
- داده‌ها به دو بخش آموزش و آزمون تقسیم شده‌اند تا بتوان عملکرد مدل را به‌درستی ارزیابی کرد.
- مدل درخت تصمیم با استفاده از داده‌های آموزشی آموزش دیده و سپس با تابع export_text، قوانین تصمیم‌گیری آن استخراج شده‌اند. این قوانین به صورت متنی نمایش داده می‌شوند و قابل فهم برای انسان هستند.
- مدل روی داده‌های آزمون پیش‌بینی انجام داده و خروجی آن با برچسب‌های واقعی مقایسه شده است.
- در نهایت، دقت مدل و گزارش طبقه‌بندی شامل معیارهایی مانند Precision، Recall و F1-Score برای هر کلاس محاسبه و نمایش داده شده‌اند.

این مثال نشان می‌دهد که چگونه می‌توان با استفاده از درخت تصمیم‌گیری، یک مدل قابل تفسیر برای طبقه‌بندی داده‌ها ساخت. این نوع مدل‌ها به‌ویژه در کاربردهایی که شفافیت و توضیح‌پذیری اهمیت دارد، بسیار مفید هستند.

القای قوانین با استفاده از الگوریتم پوشش متوالی (Rule Induction Using a Sequential Covering Algorithm)

القای قوانین شامل تولید خودکار قوانین از یک مجموعه داده بدون اتکا به یک ساختار مدل از پیش تعریف شده است. الگوریتم‌های پوشش متوالی (Sequential Covering Algorithms) دسته‌ای از تکنیک‌های القای قوانین هستند که به صورت تکراری قوانین را با انتخاب نمونه‌ها و پوشش دادن آن‌ها با قوانین، کشف می‌کنند. این فرآیند تا زمانی ادامه می‌یابد که تمام نمونه‌ها پوشش داده شوند یا یک معیار توقف برآورده شود.

مفاهیم کلیدی:

- پوشش متوالی (Sequential Covering):** الگوریتم به صورت تکراری نمونه‌هایی را که توسط قوانین موجود پوشش داده نشده‌اند انتخاب کرده و قوانینی را به طور خاص برای آن نمونه‌ها تولید می‌کند.
- معیارهای کیفیت قانون (Rule Quality Measures):** الگوریتم معمولاً از معیارهای کیفی برای ارزیابی مفید بودن یک قانون استفاده می‌کند، مانند پشتیبانی (Support)، اعتماد (Confidence) یا سود اطلاعاتی (Information Gain).
- فرآیند تکراری (Iterative Process):** این فرآیند تا زمانی ادامه می‌یابد که یک معیار توقف از پیش تعریف شده، مانند پوشش دادن تمام نمونه‌ها یا رسیدن به یک پیچیدگی قانونی خاص، برآورده شود.

مثال عملی در پایتون (Python):

یک مثال عملی با استفاده از کتابخانه "RuleFit" در پایتون (Python) را برای انجام القای قوانین با استفاده از یک الگوریتم پوشش متوالی در نظر بگیریم. RuleFit یک مدل ترکیبی است که درختان تصمیم (Decision Trees) را با مدل‌های خطی (Linear Models) ترکیب می‌کند تا قوانین قابل تفسیری را ایجاد کند.

```
In [ ]: pip install rulefit
```

```
In [ ]: pip install imodels
```

```
In [ ]: # وارد کردن کتابخانه‌های مورد نیاز
import numpy as np
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier
from imodels import RuleFitClassifier
from sklearn.multiclass import OneVsRestClassifier
from sklearn.metrics import accuracy_score, classification_report

# بارگذاری مجموعه داده آیریس
iris = datasets.load_iris()
X = iris.data # ویژگی‌ها
y = iris.target # گونه گل

# تقسیم داده‌ها به آموزش و آزمون
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Gradient Boosting تعریف مدل پایه درختی با استفاده از
tree_gen = GradientBoostingClassifier(
    n_estimators=50,
    max_depth=3,
    learning_rate=0.1,
    random_state=42
)

# با درخت پایه RuleFit ایجاد مدل
base_classifier = RuleFitClassifier(
    tree_generator=tree_gen,
    max_rules=100,
    random_state=42
```

```
)

# برای طبقه‌بندی چندکلاسه One-vs-Rest استفاده از
model = OneVsRestClassifier(base_classifier)
model.fit(X_train, y_train)

# پیش‌بینی روی داده‌های آزمون
predictions = model.predict(X_test)
accuracy = accuracy_score(y_test, predictions)
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:\n", classification_report(y_test, predictions))

# -----
# استخراج تمام قوانین (حتی با ضریب صفر)
# -----
print("\nExtracted Rules (One-vs-Rest):")
feature_names = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']

# One-vs-Rest نمایش قوانین برای هر کلاس در حالت
for i, estimator in enumerate(model.estimators_):
    print(f"\n--- Rules for class {i} vs Rest ---")

    # بررسی قابلیت استخراج قوانین
    if hasattr(estimator, 'get_rules'):
        rules = estimator.get_rules(include_thresholds=False) # دریافت همه قوانین

        # مرتب‌سازی بر اساس میزان پشتیبانی
        rules_sorted = rules.sort_values("support", ascending=False)

        if len(rules_sorted) == 0:
            print(" No rules extracted. Model may be using only intercept.")
        else:
            for _, rule in rules_sorted.iterrows():
                rule_str = rule['rule']
                # جایگزینی نام ویژگی‌ها به جای X_0, X_1, ...
                for j, name in enumerate(feature_names):
                    rule_str = rule_str.replace(f'X_{j}', name)
                coef = rule['coef']
                support = rule['support']
                print(f" Rule: {rule_str} | Coef: {coef:+.3f} | Support: {support:.3f}")

    else:
        print(" Estimator does not support rule extraction.")
```

Accuracy: 1.00

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

Extracted Rules (One-vs-Rest):

--- Rules for class 0 vs Rest ---
Estimator does not support rule extraction.

--- Rules for class 1 vs Rest ---
Estimator does not support rule extraction.

--- Rules for class 2 vs Rest ---
Estimator does not support rule extraction.



توضیحات مثال کد پایتون

در این مثال، از ترکیب **RuleFit** و **One-vs-Rest** برای ساخت یک مدل قابل تفسیر جهت طبقه‌بندی گونه‌های گل آیریس استفاده شده است. این مدل علاوه بر دقت بالا، قابلیت استخراج قوانین تصمیم‌گیری را نیز دارد:

- ابتدا داده‌های مجموعه **Iris** بارگذاری شده‌اند که شامل چهار ویژگی عددی برای سه گونه گل است.
- داده‌ها به دو بخش آموزش و آزمون تقسیم شده‌اند تا عملکرد مدل به‌درستی ارزیابی شود.
- مدل پایه از نوع **Gradient Boosting** انتخاب شده که درخت‌های قدرتمند و دقیق تولید می‌کند.
- با استفاده از **RuleFitClassifier**، قوانین تصمیم‌گیری از درخت‌ها استخراج شده و به مدل اضافه می‌شوند. این قوانین به صورت شرط‌های ساده قابل فهم هستند.
- برای پشتیبانی از طبقه‌بندی چندکلاسه، از **One-vs-Rest** استفاده شده تا برای هر کلاس یک مدل جداگانه ساخته شود.
- در پایان، دقت مدل و گزارش طبقه‌بندی نمایش داده شده و تمام قوانین استخراج‌شده برای هر کلاس به صورت کامل چاپ شده‌اند.

این مثال نشان می‌دهد که چگونه می‌توان با استفاده از **مدل‌های مبتنی بر قانون**، هم به دقت بالا دست یافت و هم شفافیت و توضیح‌پذیری مدل را حفظ کرد. چنین رویکردهایی در کاربردهای حساس مانند پزشکی، حقوق یا مالی بسیار ارزشمند هستند.

7.4.4. Associative classification

طبقه‌بندی انجمنی (Associative Classification) ⚖️

طبقه‌بندی انجمنی، که به عنوان طبقه‌بندی مبتنی بر قوانین (rule-based classification) یا طبقه‌بندی با انجمن (classification by association) نیز شناخته می‌شود، اصول استخراج قوانین انجمنی (association rule mining) را با طبقه‌بندی سنتی ترکیب می‌کند. به جای تولید قوانین به صورت جداگانه، طبقه‌بندی انجمنی به صورت همزمان قوانین انجمنی را کشف کرده و از آن‌ها برای مقاصد طبقه‌بندی استفاده می‌کند. این رویکرد اغلب از تکنیک‌هایی مانند الگوریتم Apriori برای استخراج مجموعه‌های مکرر آیتم (frequent itemsets) و یک طبقه‌بندی‌کننده (classifier) برای تولید قوانین بر اساس این مجموعه‌ها بهره می‌برد.

مفاهیم کلیدی:

- استخراج قوانین انجمنی (Association Rule Mining):** شناسایی الگوها یا انجمن‌های مکرر بین متغیرها در مجموعه داده.
- تولید قوانین (Rule Generation):** استخراج قوانین طبقه‌بندی از مجموعه‌های مکرر آیتم که در طول استخراج قوانین انجمنی کشف شده‌اند.
- رویکرد ترکیبی (Hybrid Approach):** ادغام نقاط قوت هر دو استخراج قوانین انجمنی و طبقه‌بندی برای افزایش عملکرد پیش‌بینی‌کننده مدل.

مثال عملی در پایتون (Python):

در این مثال عملی، از کتابخانه **mlxtend** برای استخراج قوانین انجمنی و استفاده از آن‌ها در طبقه‌بندی داده‌ها استفاده می‌کنیم. ابتدا داده‌های عددی به بازه‌های گسسته تقسیم شده و سپس با **One-Hot Encoding** به فرمت 0/1 تبدیل می‌شوند تا قابلیت استفاده در الگوریتم **Apriori** و استخراج **Association Rules** فراهم شود. در نهایت، قوانین استخراج شده برای پیش‌بینی کلاس هدف (Target) مورد استفاده قرار می‌گیرند.

```
In [ ]: pip install scikit-learn
```

```
In [ ]: # وارد کردن کتابخانه‌های مورد نیاز
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import KBinsDiscretizer, OneHotEncoder
from mlxtend.frequent_patterns import apriori, association_rules
import pandas as pd
import numpy as np
from sklearn.metrics import accuracy_score, classification_report
```

```
# بارگذاری مجموعه داده آیریس
iris = datasets.load_iris()
X = iris.data # ویژگی‌ها
y = iris.target # برچسب‌ها

# برای پردازش راحت‌تر DataFrame تبدیل داده‌ها به
df = pd.DataFrame(X, columns=iris.feature_names)
df['target'] = y # افزودن ستون برچسب‌ها

# گسسته‌سازی ویژگی‌های عددی به ۳ بازه مساوی
kbd = KBinsDiscretizer(n_bins=3, encode='ordinal', strategy='uniform')
X_binned = kbd.fit_transform(df[iris.feature_names])
df_binned = pd.DataFrame(X_binned, columns=iris.feature_names).astype(int)
df_binned['target'] = df['target'] # افزودن برچسب‌ها به داده‌های گسسته‌شده

# برای تحلیل انجمنی One-Hot تبدیل ویژگی‌ها و برچسب‌ها به قالب
ohe = OneHotEncoder(sparse_output=False)
ohe_data = ohe.fit_transform(df_binned)
df_ohe = pd.DataFrame(ohe_data, columns=ohe.get_feature_names_out(df_binned.columns))

# تقسیم داده‌ها به آموزش و آزمون
df_train, df_test = train_test_split(df_ohe, test_size=0.2, random_state=42)

# استخراج مجموعه‌های پرتکرار از داده‌های آموزشی با الگوریتم Apriori
frequent_itemsets = apriori(df_train, min_support=0.2, use_colnames=True)

# تولید قوانین انجمنی از مجموعه‌های پرتکرار
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.7)

# فیلتر کردن قوانینی که نتیجه‌شان شامل برچسب هدف باشد
target_columns = [c for c in df_ohe.columns if c.startswith('target')]
target_rules = rules[rules['consequents'].apply(lambda x: any(tc in str(x) for tc in target_columns))]

# تابع پیش‌بینی برای یک ردیف با استفاده از قوانین استخراج‌شده
def predict_row(row, rules_df):
    for _, rule in rules_df.iterrows():
        antecedents = set(rule['antecedents'])
        if antecedents.issubset(set(row.index[row.astype(bool)])):
            # استخراج مقدار هدف از نتیجه قانون
            for tc in target_columns:
                if tc in rule['consequents']:
                    return int(tc.split('_')[-1])
    # اگر هیچ قانونی اعمال نشد، بازگشت به کلاس پرتکرار در داده‌های آموزشی
    return df_binned['target'].mode()[0]

# پیش‌بینی برچسب‌ها برای داده‌های آزمون
y_test = df_binned.iloc[df_test.index]['target'].astype(int)
y_pred = df_test.apply(lambda row: predict_row(row, target_rules), axis=1)

# ارزیابی مدل با دقت و گزارش طبقه‌بندی
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:\n", classification_report(y_test, y_pred))

# نمایش قوانین استخراج‌شده
print("\nExtracted Rules:")
for _, rule in target_rules.iterrows():
    print(f"IF {rule['antecedents']} THEN {rule['consequents']} (support={rule['support']:.2f}, confidence={rule['confidence']:.2f})")
```

Accuracy: 0.97

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	0.90	1.00	0.95	9
2	1.00	0.91	0.95	11
accuracy			0.97	30
macro avg	0.97	0.97	0.97	30
weighted avg	0.97	0.97	0.97	30

Extracted Rules:

IF frozenset({'sepal length (cm)_0'}) THEN frozenset({'target_0'}) (support=0.31, confidence=0.84)

IF frozenset({'petal length (cm)_0'}) THEN frozenset({'target_0'}) (support=0.33, confidence=1.00)

IF frozenset({'petal length (cm)_1'}) THEN frozenset({'target_1'}) (support=0.33, confidence=0.89)

IF frozenset({'petal length (cm)_2'}) THEN frozenset({'target_2'}) (support=0.28, confidence=0.94)

IF frozenset({'petal width (cm)_0'}) THEN frozenset({'target_0'}) (support=0.33, confidence=1.00)

IF frozenset({'petal width (cm)_1'}) THEN frozenset({'target_1'}) (support=0.33, confidence=0.91)

IF frozenset({'petal width (cm)_2'}) THEN frozenset({'target_2'}) (support=0.29, confidence=0.95)

IF frozenset({'sepal length (cm)_0', 'sepal width (cm)_1'}) THEN frozenset({'target_0'}) (support=0.23, confidence=0.97)

IF frozenset({'sepal length (cm)_0', 'petal length (cm)_0'}) THEN frozenset({'target_0'}) (support=0.31, confidence=1.00)

IF frozenset({'sepal length (cm)_0'}) THEN frozenset({'target_0', 'petal length (cm)_0'}) (support=0.31, confidence=0.84)

IF frozenset({'petal length (cm)_0'}) THEN frozenset({'sepal length (cm)_0', 'target_0'}) (support=0.31, confidence=0.93)

IF frozenset({'sepal length (cm)_0', 'petal width (cm)_0'}) THEN frozenset({'target_0'}) (support=0.31, confidence=1.00)

IF frozenset({'sepal length (cm)_0'}) THEN frozenset({'target_0', 'petal width (cm)_0'}) (support=0.31, confidence=0.84)

IF frozenset({'petal width (cm)_0'}) THEN frozenset({'sepal length (cm)_0', 'target_0'}) (support=0.31, confidence=0.93)

IF frozenset({'sepal length (cm)_1', 'petal length (cm)_1'}) THEN frozenset({'target_1'}) (support=0.24, confidence=0.88)

IF frozenset({'sepal length (cm)_1', 'petal width (cm)_1'}) THEN frozenset({'target_1'}) (support=0.24, confidence=0.91)

IF frozenset({'sepal width (cm)_1', 'petal length (cm)_0'}) THEN frozenset({'target_0'}) (support=0.23, confidence=1.00)

IF frozenset({'petal length (cm)_0'}) THEN frozenset({'target_0', 'sepal width (cm)_1'}) (support=0.23, confidence=0.70)

IF frozenset({'petal width (cm)_0', 'sepal width (cm)_1'}) THEN frozenset({'target_0'}) (support=0.23, confidence=1.00)

IF frozenset({'petal width (cm)_0'}) THEN frozenset({'target_0', 'sepal width (cm)_1'}) (support=0.23, confidence=0.70)

IF frozenset({'petal width (cm)_0', 'petal length (cm)_0'}) THEN frozenset({'target_0'}) (support=0.33, confidence=1.00)

IF frozenset({'petal width (cm)_0'}) THEN frozenset({'target_0', 'petal length (cm)_0'}) (support=0.33, confidence=1.00)

IF frozenset({'petal length (cm)_0'}) THEN frozenset({'target_0', 'petal width (cm)_0'}) (support=0.33, confidence=1.00)

IF frozenset({'petal length (cm)_1', 'petal width (cm)_1'}) THEN frozenset({'target_1'}) (support=0.32, confidence=1.00)

IF frozenset({'petal length (cm)_1'}) THEN frozenset({'target_1', 'petal width (cm)_1'}) (support=0.32, confidence=0.86)

IF frozenset({'petal width (cm)_1'}) THEN frozenset({'target_1', 'petal length (cm)_1'}) (support=0.32, confidence=0.88)

IF frozenset({'petal length (cm)_2', 'petal width (cm)_2'}) THEN frozenset({'target_2'}) (support=0.25, confidence=0.97)

IF frozenset({'petal length (cm)_2'}) THEN frozenset({'petal width (cm)_2', 'target_2'}) (support=0.25, confidence=0.83)

IF frozenset({'petal width (cm)_2'}) THEN frozenset({'petal length (cm)_2', 'target_2'}) (support=0.25, confidence=0.81)

IF frozenset({'sepal length (cm)_0', 'sepal width (cm)_1', 'petal length (cm)_0'}) THEN frozenset({'target_0'}) (support=0.23, confidence=1.00)

IF frozenset({'sepal length (cm)_0', 'petal length (cm)_0'}) THEN frozenset({'target_0', 'sepal width (cm)_1'}) (support=0.23, confidence=0.76)

IF frozenset({'sepal length (cm)_0', 'sepal width (cm)_1'}) THEN frozenset({'target_0', 'petal length (cm)_0'}) (support=0.23, confidence=0.97)

IF frozenset({'sepal width (cm)_1', 'petal length (cm)_0'}) THEN frozenset({'sepal length (cm)_0', 'target_0'}) (support=0.23, confidence=1.00)

IF frozenset({'petal length (cm)_0'}) THEN frozenset({'sepal length (cm)_0', 'target_0', 'sepal width (cm)_1'}) (support=0.23, confidence=0.70)

IF frozenset({'sepal length (cm)_0', 'petal width (cm)_0', 'sepal width (cm)_1'}) THEN frozenset({'target_0'}) (support=0.23, confidence=1.00)

IF frozenset({'sepal length (cm)_0', 'petal width (cm)_0'}) THEN frozenset({'target_0', 'sepal width (cm)_1'}) (support=0.23, confidence=0.76)

IF frozenset({'sepal length (cm)_0', 'sepal width (cm)_1'}) THEN frozenset({'target_0', 'petal width (cm)_0'}) (support=0.23, confidence=0.97)

IF frozenset({'petal width (cm)_0', 'sepal width (cm)_1'}) THEN frozenset({'sepal length (cm)_0', 'target_0'}) (support=0.23, confidence=1.00)

IF frozenset({'petal width (cm)_0'}) THEN frozenset({'sepal length (cm)_0', 'target_0', 'sepal width (cm)_1'}) (support=0.23, confidence=0.70)

IF frozenset({'sepal length (cm)_0', 'petal width (cm)_0', 'petal length (cm)_0'}) THEN frozenset({'target_0'}) (support=0.31, confidence=1.00)

IF frozenset({'sepal length (cm)_0', 'petal width (cm)_0'}) THEN frozenset({'target_0', 'petal length (cm)_0'}) (support=0.31, confidence=1.00)

IF frozenset({'sepal length (cm)_0', 'petal length (cm)_0'}) THEN frozenset({'target_0', 'petal width (cm)_0'}) (support=0.31, confidence=1.00)

IF frozenset({'petal width (cm)_0', 'petal length (cm)_0'}) THEN frozenset({'sepal length (cm)_0', 'target_0'}) (support=0.31, confidence=0.93)

IF frozenset({'sepal length (cm)_0'}) THEN frozenset({'target_0', 'petal width (cm)_0', 'petal length (cm)_0'}) (support=0.31, confidence=0.84)

IF frozenset({'petal width (cm)_0'}) THEN frozenset({'sepal length (cm)_0', 'target_0', 'petal length (cm)_0'}) (support=0.31, confidence=0.93)

IF frozenset({'petal length (cm)_0'}) THEN frozenset({'sepal length (cm)_0', 'petal width (cm)_0', 'target_0'}) (support=0.31, confidence=0.93)

IF frozenset({'sepal length (cm)_1', 'petal length (cm)_1', 'petal width (cm)_1'}) THEN frozenset({'target_1'}) (support=0.23, confidence=1.00)

IF frozenset({'sepal length (cm)_1', 'petal length (cm)_1'}) THEN frozenset({'target_1', 'petal width (cm)_1'}) (support=0.23, confidence=0.85)

IF frozenset({'sepal length (cm)_1', 'petal width (cm)_1'}) THEN frozenset({'target_1', 'petal length (cm)_1'}) (support=0.23, confidence=0.88)

IF frozenset({'petal length (cm)_1', 'petal width (cm)_1'}) THEN frozenset({'sepal length (cm)_1', 'target_1'}) (support=0.23, confidence=0.74)

IF frozenset({'sepal width (cm)_1', 'petal width (cm)_0', 'petal length (cm)_0'}) THEN frozenset({'target_0'}) (support=0.23, confidence=1.00)

IF frozenset({'petal width (cm)_0', 'petal length (cm)_0'}) THEN frozenset({'target_0', 'sepal width (cm)_1'}) (support=0.23, confidence=0.70)

IF frozenset({'petal width (cm)_0', 'sepal width (cm)_1'}) THEN frozenset({'target_0', 'petal length (cm)_0'}) (support=0.23, confidence=1.00)

IF frozenset({'sepal width (cm)_1', 'petal length (cm)_0'}) THEN frozenset({'target_0', 'petal width (cm)_0'}) (support=0.23, confidence=1.00)

IF frozenset({'petal width (cm)_0'}) THEN frozenset({'target_0', 'sepal width (cm)_1', 'petal length (cm)_0'}) (support=0.23, confidence=0.70)

IF frozenset({'petal length (cm)_0'}) THEN frozenset({'target_0', 'petal width (cm)_0', 'sepal width (cm)_1'}) (support=0.23, confidence=0.70)

IF frozenset({'sepal length (cm)_0', 'petal width (cm)_0', 'petal length (cm)_0', 'sepal width (cm)_1'}) THEN frozenset({'target_0'}) (support=0.23, confidence=1.00)

IF frozenset({'sepal length (cm)_0', 'petal width (cm)_0', 'sepal width (cm)_1'}) THEN frozenset({'target_0', 'petal length (cm)_0'}) (support=0.23, confidence=1.00)

IF frozenset({'sepal length (cm)_0', 'petal length (cm)_0', 'sepal width (cm)_1'}) THEN frozenset({'target_0', 'petal width (cm)_0'}) (support=0.23, confidence=1.00)

IF frozenset({'petal width (cm)_0', 'petal length (cm)_0', 'sepal width (cm)_1'}) THEN frozenset({'target_0', 'sepal length (cm)_0'}) (support=0.23, confidence=1.00)

IF frozenset({'sepal length (cm)_0', 'petal width (cm)_0', 'petal length (cm)_0'}) THEN frozenset({'target_0', 'sepal width (cm)_1'}) (support=0.23, confidence=0.76)

IF frozenset({'sepal length (cm)_0', 'sepal width (cm)_1'}) THEN frozenset({'target_0', 'petal width (cm)_0', 'petal length (cm)_0'}) (support=0.23, confidence=0.97)

IF frozenset({'petal width (cm)_0', 'sepal width (cm)_1'}) THEN frozenset({'target_0', 'sepal length (cm)_0', 'petal length (cm)_0'}) (support=0.23, confidence=1.00)

IF frozenset({'petal length (cm)_0', 'sepal width (cm)_1'}) THEN frozenset({'target_0', 'petal width (cm)_0', 'sepal length (cm)_0'}) (support=0.23, confidence=1.00)

IF frozenset({'sepal length (cm)_0', 'petal width (cm)_0'}) THEN frozenset({'target_0', 'petal length (cm)_0', 'sepal width (cm)_1'}) (support=0.23, confidence=0.76)

IF frozenset({'sepal length (cm)_0', 'petal length (cm)_0'}) THEN frozenset({'target_0', 'petal width (cm)_0', 'sepal width (cm)_1'}) (support=0.23, confidence=0.76)

IF frozenset({'petal width (cm)_0', 'petal length (cm)_0'}) THEN frozenset({'target_0', 'sepal length (cm)_0', 'sepal width (cm)_1'}) (support=0.23, confidence=0.70)

IF frozenset({'petal width (cm)_0'}) THEN frozenset({'target_0', 'sepal length (cm)_0', 'petal length (cm)_0', 'sepal width (cm)_1'}) (support=0.23, confidence=0.70)

توضیحات مثال کد پایتون

در این مثال، از ترکیب تحلیل انجمنی (Association Analysis) و قوانین پرتکرار برای طبقه‌بندی گونه‌های گل آیریس استفاده شده است. این روش برخلاف مدل‌های یادگیری سنتی، بر پایه کشف الگوهای پرتکرار در داده‌ها عمل می‌کند:

- ابتدا داده‌های عددی مجموعه Iris به سه بازه مساوی گسسته‌سازی شده‌اند تا برای تحلیل انجمنی مناسب شوند.
 - سپس تمام ویژگی‌ها و برچسب‌ها به قالب One-Hot Encoding تبدیل شده‌اند تا بتوان از آن‌ها در الگوریتم Apriori استفاده کرد.
 - با استفاده از الگوریتم Apriori، مجموعه‌های پرتکرار استخراج شده‌اند و سپس قوانین انجمنی با حداقل اطمینان ۰.۷ تولید شده‌اند.
 - فقط قوانینی که نتیجه‌شان شامل برچسب هدف (target) باشد نگه داشته شده‌اند تا برای پیش‌بینی استفاده شوند.
 - برای هر نمونه آزمون، بررسی می‌شود که آیا پیش‌شرط‌های یک قانون در آن وجود دارد یا نه؛ اگر وجود داشته باشد، مقدار هدف از آن قانون استخراج می‌شود. در غیر این صورت، کلاس پرتکرار به عنوان پیش‌بینی انتخاب می‌شود.
 - در پایان، دقت مدل و گزارش طبقه‌بندی محاسبه شده و همچنین قوانین استخراج‌شده به صورت متنی نمایش داده شده‌اند.
- این مثال نشان می‌دهد که چگونه می‌توان از قوانین انجمنی برای ساخت مدل‌های طبقه‌بندی استفاده کرد؛ روشی که به‌ویژه در تحلیل‌های توصیفی و کاربردهایی با نیاز به شفافیت و توضیح‌پذیری بالا بسیار مؤثر است.

7.4.5. Discriminative frequent pattern-based classification

طبقه‌بندی مبتنی بر الگوهای مکرر تفکیک‌کننده (Discriminative frequent pattern-based classification)

طبقه‌بندی مبتنی بر الگوهای مکرر تفکیک‌کننده (Discriminative frequent pattern-based classification) رویکردی است که بر یافتن الگوهای مکرری تمرکز دارد که ارتباط بسیار بالایی با کلاس‌های خاص در مجموعه داده دارند. به جای تولید قوانین بر اساس انجمن‌ها در کل مجموعه داده، این روش به دنبال کشف الگوهایی است که به طور مؤثر بین کلاس‌های مختلف تمایز قائل می‌شوند.

مفاهیم کلیدی:

- استخراج الگوی مکرر (Frequent Pattern Mining): شناسایی الگوهایی که به طور مکرر در مجموعه داده رخ می‌دهند.
- الگوهای تفکیک‌کننده (Discriminative Patterns): الگوهایی که تفاوت‌های قابل توجهی در وقوع بین کلاس‌های مختلف نشان می‌دهند.

- طبقه‌بندی بر اساس الگوهای تفکیک‌کننده (Classification based on Discriminative Patterns): استفاده از

الگوهای تفکیک‌کننده کشف شده برای ساخت یک طبقه‌بندی‌کننده (classifier) که می‌تواند به طور مؤثر بین کلاس‌ها

تمایز قائل شود.

مثال عملی در پایتون (Python):

یک مثال عملی برای طبقه‌بندی مبتنی بر الگوهای مکرر تفکیک‌کننده (Discriminative Frequent Pattern Classification) را در

نظر بگیریم. در این مثال از کتابخانه **mlxtend** استفاده شده و الگوریتم **Apriori + Association Rules** برای استخراج قوانین

الگوهای مکرر به کار رفته است. ویژگی‌های عددی ابتدا به دسته‌های متعدد تقسیم (binning) شده‌اند تا قوانین دقیق‌تری ایجاد

شوند.

In []:

```
import pandas as pd
from sklearn import datasets
from sklearn.model_selection import train_test_split
from mlxtend.frequent_patterns import apriori, association_rules
from sklearn.metrics import accuracy_score, classification_report

# بارگذاری مجموعه داده آیریس
iris = datasets.load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target # اضافه کردن ستون برچسب کلاس

# با ۷ دسته (binning) تبدیل ویژگی‌ها به بازه‌های گسسته
for col in iris.feature_names:
    df[col] = pd.qcut(df[col], q=7, labels=[0,1,2,3,4,5,6])

# تقسیم داده‌ها به آموزش و آزمون با حفظ نسبت کلاس‌ها
df_train, df_test = train_test_split(df, test_size=0.2, random_state=42, stratify=df['target'])

# برای الگوریتم one-hot تبدیل داده‌ها به قالب
df_train_ohe = pd.get_dummies(df_train.astype(str))
df_test_ohe = pd.get_dummies(df_test.astype(str))
df_test_ohe = df_test_ohe.reindex(columns=df_train_ohe.columns, fill_value=0) # هم‌ترازی ستون‌ها
df_train_ohe = df_train_ohe.astype(bool)
df_test_ohe = df_test_ohe.astype(bool)

# استخراج مجموعه‌های پرتکرار و قوانین انجمنی
frequent_itemsets = apriori(df_train_ohe, min_support=0.05, use_colnames=True)
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.5)

# فیلتر کردن قوانینی که نتیجه‌شان شامل کلاس هدف باشد
rules = rules[rules['consequents'].apply(lambda x: any('target_' in i for i in x))]
rules = rules.sort_values(by=['confidence', 'support'], ascending=False)

# تعیین کلاس پیش‌فرض در صورت عدم تطابق با هیچ قانون
major_class = df_train['target'].mode()[0]

# تابع طبقه‌بندی بر اساس قوانین استخراج‌شده
def classify(transaction, rules):
    for _, row in rules.iterrows():
        if row['antecedents'].issubset(transaction):
            for item in row['consequents']:
                if 'target_' in item:
                    return int(item.split('_')[1])
    return major_class # بازگشت به کلاس پیش‌فرض در صورت عدم تطابق

# آماده‌سازی تراکنش‌ها از داده‌های آزمون
X_test_transactions = [set(df_test_ohe.iloc[i][df_test_ohe.columns != 'target'].index) for i in range(len(df_test_ohe))]

# پیش‌بینی کلاس‌ها با استفاده از قوانین
predictions = [classify(tx, rules) for tx in X_test_transactions]

# ارزیابی عملکرد مدل
accuracy = accuracy_score(df_test['target'], predictions)
print(f"Accuracy: {accuracy:.2f}")
print(classification_report(df_test['target'], predictions, zero_division=0))

# نمایش قوانین استخراج‌شده
print("\nExtracted Rules:")
for _, row in rules.iterrows():
    print(f"IF {row['antecedents']} THEN {row['consequents']} (support={row['support']:.2f}, confidence={row['confidence']:.2f})")
```

	precision	recall	f1-score	support
0	0.33	1.00	0.50	10
1	0.00	0.00	0.00	10
2	0.00	0.00	0.00	10
accuracy			0.33	30
macro avg	0.11	0.33	0.17	30
weighted avg	0.11	0.33	0.17	30

[illegible]

IF frozenset({'sepal length (cm)_0', 'petal width (cm)_0'}) THEN frozenset({'target_0', 'petal length (cm)_0'}) (support=0.06, confidence=0.50)
IF frozenset({'petal width (cm)_2'}) THEN frozenset({'petal length (cm)_2', 'target_1', 'sepal width (cm)_0'}) (support=0.06, confidence=0.50)
IF frozenset({'petal width (cm)_0', 'petal length (cm)_1'}) THEN frozenset({'sepal length (cm)_0', 'target_0'}) (support=0.05, confidence=0.50)

توضیحات مثال کد پایتون

در این مثال، از الگوریتم **Apriori** برای استخراج قوانین انجمنی و ساخت یک طبقه‌بند ساده مبتنی بر قانون استفاده شده است. این رویکرد به جای مدل‌های عددی، از منطق شرطی برای تصمیم‌گیری بهره می‌برد:

- ابتدا داده‌های مجموعه **Iris** بارگذاری شده و ویژگی‌های عددی آن به صورت گسسته (binning) در ۷ دسته تقسیم شده‌اند تا برای تحلیل انجمنی مناسب شوند.
 - داده‌ها به قالب **One-Hot Encoding** تبدیل شده‌اند تا بتوان از الگوریتم Apriori برای استخراج الگوهای پرتکرار استفاده کرد.
 - با استفاده از تابع **association_rules**، قوانینی استخراج شده‌اند که نتیجه آن‌ها شامل کلاس هدف (target) باشد.
 - یک طبقه‌بند ساده تعریف شده که برای هر نمونه آزمون، بررسی می‌کند آیا قوانین موجود با ویژگی‌های آن تطابق دارند یا خیر. در صورت تطابق، کلاس متناظر با قانون اعمال می‌شود؛ در غیر این صورت، از کلاس پرتکرار به عنوان پیش‌فرض استفاده می‌شود.
 - در پایان، دقت مدل و گزارش طبقه‌بندی نمایش داده شده و تمام قوانین استخراج‌شده به صورت خوانا چاپ شده‌اند.
- این مثال نشان می‌دهد که چگونه می‌توان با استفاده از **تحلیل انجمنی**، یک مدل طبقه‌بندی قابل تفسیر و ساده ساخت. چنین مدل‌هایی در کاربردهایی که شفافیت و قابلیت توضیح اهمیت دارد، بسیار مفید هستند.

Section 7.5

7.5.1. Semisupervised classification

طبقه‌بندی نیمه‌نظارتی (Semi-supervised Classification)

طبقه‌بندی نیمه‌نظارتی (Semi-supervised Classification) نوعی از یادگیری با نظارت ضعیف (weakly supervised learning) است که در آن مجموعه داده آموزشی شامل نمونه‌های برچسب‌دار و بدون برچسب است. یادگیری نظارت‌شده سنتی (supervised learning) برای آموزش بر داده‌های برچسب‌دار اتکا دارد، در حالی که یادگیری بدون نظارت (unsupervised learning) با داده‌های بدون برچسب سروکار دارد. یادگیری نیمه‌نظارتی با استفاده از مقدار کمی داده برچسب‌دار همراه با مقدار بیشتری داده بدون برچسب، به دنبال بهره‌برداری از مزایای هر دو برای ساخت یک مدل قوی‌تر است.

مفاهیم کلیدی:

- داده‌های برچسب‌دار و بدون برچسب (Labeled and Unlabeled Data):** مجموعه داده آموزشی شامل نمونه‌هایی با برچسب‌های مشخص (برچسب‌دار) و نمونه‌هایی بدون برچسب (بدون برچسب) است.

- بهره‌برداری از داده‌های بدون برچسب (Leveraging Unlabeled Data): از داده‌های بدون برچسب برای بهبود تعمیم و عملکرد طبقه‌بندی‌کننده (classifier) استفاده می‌شود.
- تکنیک‌های رایج (Common Techniques): خودآموزی (Self-training)، آموزش مشترک (co-training) و یادگیری چندنمایی (multi-view learning) تکنیک‌های رایج یادگیری نیمه‌نظارتی هستند.

مثال عملی در پایتون (Python):

یک مثال عملی با استفاده از کتابخانه "scikit-learn" در پایتون (Python) برای طبقه‌بندی نیمه‌نظارتی را در نظر بگیریم. ما از یک مجموعه داده ساده و یک الگوریتم نیمه‌نظارتی معروف به انتشار برچسب (Label Propagation) استفاده خواهیم کرد.

```
In [ ]: # وارد کردن کتابخانه‌های مورد نیاز
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.semi_supervised import LabelPropagation
from sklearn.metrics import accuracy_score, classification_report

# بارگذاری مجموعه داده آیریس
iris = datasets.load_iris()
X = iris.data # ویژگی‌ها
y = iris.target # برچسب‌ها

# اعمال یادگیری نیمه‌نظارتی با حذف تصادفی بخشی از برچسب‌ها
import numpy as np
rng = np.random.RandomState(42)
y_semi_supervised = y.copy()
y_semi_supervised[rng.rand(len(y)) < 0.5] = -1 # مقدار 1- نشان‌دهنده داده بدون برچسب است

# تقسیم داده‌ها به آموزش و آزمون
X_train, X_test, y_train, y_test = train_test_split(X, y_semi_supervised, test_size=0.2, random_state=42)

# ایجاد مدل یادگیری نیمه‌نظارتی با الگوریتم انتشار برچسب (Label Propagation)
label_propagation_classifier = LabelPropagation(kernel="knn", n_neighbors=10)

# آموزش مدل روی داده‌های آموزشی شامل نمونه‌های بدون برچسب
label_propagation_classifier.fit(X_train, y_train)

# انجام پیش‌بینی روی داده‌های آزمون
predictions = label_propagation_classifier.predict(X_test)

# ارزیابی مدل فقط روی نمونه‌هایی که برچسب واقعی دارند
accuracy = accuracy_score(y_test[y_test != -1], predictions[y_test != -1])
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:\n", classification_report(y_test[y_test != -1], predictions[y_test != -1]))
```

Accuracy: 1.00

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	4
1	1.00	1.00	1.00	4
2	1.00	1.00	1.00	3
accuracy			1.00	11
macro avg	1.00	1.00	1.00	11
weighted avg	1.00	1.00	1.00	11



توضیحات مثال کد پایتون

در این مثال، از الگوریتم انتشار برچسب (Label Propagation) برای پیاده‌سازی یک مدل یادگیری نیمه‌نظارتی روی مجموعه داده آیریس استفاده شده است. این نوع یادگیری زمانی کاربرد دارد که فقط بخشی از داده‌ها دارای برچسب باشند:

- ابتدا مجموعه داده Iris بارگذاری شده که شامل ویژگی‌های عددی برای سه گونه گل مختلف است.
- سپس به صورت تصادفی، نیمی از برچسب‌ها حذف شده‌اند (با مقدار -1) تا شرایط یادگیری نیمه‌نظارتی شبیه‌سازی شود.
- داده‌ها به دو بخش آموزش و آزمون تقسیم شده‌اند و مدل Label Propagation با هسته knn و تعداد همسایه ۱۰ ساخته شده است.

- این مدل قادر است با استفاده از ساختار داده‌ها، برچسب‌های گمشده را از طریق انتشار اطلاعات از نمونه‌های برچسب‌دار به نمونه‌های بدون برچسب تخمین بزند.
- در مرحله ارزیابی، فقط نمونه‌هایی که برچسب واقعی دارند در نظر گرفته شده‌اند تا دقت و عملکرد مدل به‌درستی سنجیده شود.

این مثال نشان می‌دهد که چگونه می‌توان با استفاده از **یادگیری نیمه‌نظارتی**، از داده‌های ناقص نیز برای ساخت مدل‌های طبقه‌بندی استفاده کرد. این روش در کاربردهایی که جمع‌آوری برچسب پرهزینه یا دشوار است، بسیار مؤثر و کاربردی است.

7.5.2. Active learning

یادگیری فعال (Active Learning) 🤖

یادگیری فعال (Active Learning) یک رویکرد یادگیری نیمه‌نظارتی (semi-supervised) است که در آن الگوریتم با یک "oracle" یا یک برچسب‌زن انسانی (human annotator) تعامل می‌کند تا به صورت هوشمند برای نمونه‌هایی که آن‌ها را آموزنده‌ترین می‌داند، برچسب‌ها را درخواست کند. به جای دریافت منفعلانه نمونه‌های برچسب‌دار، الگوریتم به صورت فعال انتخاب می‌کند که برای کدام نمونه‌ها برچسب درخواست کند و هدفش به حداکثر رساندن کارایی یادگیری با حداقل تعداد نمونه‌های برچسب‌دار است.

مفاهیم کلیدی:

- **استراتژی پرس‌وجو (Query Strategy):** الگوریتم از یک استراتژی پرس‌وجو (Query Strategy) برای انتخاب نمونه‌هایی استفاده می‌کند که انتظار می‌رود آموزنده‌ترین اطلاعات را درباره مدل اصلی فراهم کنند.
- **عدم قطعیت مدل (Model Uncertainty):** نمونه‌هایی با پیش‌بینی‌های نامطمئن یا آن‌هایی که نزدیک به مرز تصمیم‌گیری (decision boundary) هستند، اغلب برای برچسب‌گذاری در اولویت قرار می‌گیرند.
- **کاهش هزینه‌های برچسب‌گذاری (Reducing Annotation Costs):** یادگیری فعال (Active Learning) با تمرکز بر نمونه‌های آموزنده، هدفش کاهش نیاز به مجموعه‌های داده برچسب‌دار بزرگ است و به همین دلیل به ویژه در مواردی که به دست آوردن داده‌های برچسب‌دار گران یا زمان‌بر است، مفید می‌باشد.

مثال عملی در پایتون (Python):

یک مثال عملی با استفاده از کتابخانه **ALiPy** در پایتون (Python 3.11) برای یادگیری فعال (Active Learning) در نظر بگیریم. در این مثال، ما از یک مجموعه داده مصنوعی ساده (synthetic dataset) و یک طبقه‌بندی‌کننده پایه (Random Forest) استفاده می‌کنیم تا فرآیند انتخاب نمونه‌های نامطمئن (uncertainty sampling) و به‌روزرسانی مدل را نشان دهیم.

```
In [ ]: import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# --- تولید داده‌های مصنوعی برای طبقه‌بندی ---
X, y = make_classification(n_samples=1000, n_features=20, n_classes=2, random_state=42)

# --- (pool) تقسیم داده‌ها به مجموعه آموزش اولیه و مجموعه استخر ---
X_train, X_pool, y_train, y_pool = train_test_split(X, y, test_size=0.95, random_state=42, stratify=y)

# --- مقداردهی اولیه مدل پایه (Base Learner) ---
learner = RandomForestClassifier(random_state=42)
learner.fit(X_train, y_train)

# --- (Uncertainty Sampling) تابع انتخاب فعال نمونه‌ها با استفاده از عدم قطعیت ---
def uncertainty_sampling(classifier, X_pool):
    """
    انتخاب نمونه‌ای که بیشترین عدم قطعیت را دارد
    عدم قطعیت = 1 - بیشترین احتمال پیش‌بینی‌شده
    """
    proba = classifier.predict_proba(X_pool)
    uncertainty = 1 - np.max(proba, axis=1)
    return np.argmax(uncertainty)

# --- (Active Learning) حلقه یادگیری فعال ---
n_queries = 50
for i in range(n_queries):
    # انتخاب نمونه‌ای که مدل بیشترین شک را نسبت به آن دارد
    query_idx = uncertainty_sampling(learner, X_pool)

    # افزودن نمونه انتخاب‌شده به مجموعه آموزش
    X_train = np.vstack([X_train, X_pool[query_idx].reshape(1, -1)])
    y_train = np.append(y_train, y_pool[query_idx])

    # حذف نمونه انتخاب‌شده از مجموعه استخر
    X_pool = np.delete(X_pool, query_idx, axis=0)
    y_pool = np.delete(y_pool, query_idx)

    # آموزش مجدد مدل با داده‌های جدید
    learner.fit(X_train, y_train)

# --- ارزیابی مدل روی کل داده‌ها ---
predictions = learner.predict(X)
accuracy = accuracy_score(y, predictions)
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:\n", classification_report(y, predictions))
```

Accuracy: 0.90

Classification Report:

	precision	recall	f1-score	support
0	0.88	0.92	0.90	500
1	0.92	0.87	0.90	500
accuracy			0.90	1000
macro avg	0.90	0.90	0.90	1000
weighted avg	0.90	0.90	0.90	1000

توضیحات مثال کد پایتون

در این مثال، یک فرآیند یادگیری فعال (Active Learning) با استفاده از مدل جنگل تصادفی (Random Forest) پیاده‌سازی شده است. هدف این رویکرد، بهبود عملکرد مدل با انتخاب هوشمندانه‌ترین نمونه‌ها برای آموزش است:

- ابتدا یک مجموعه داده مصنوعی با ۱۰۰۰ نمونه و ۲۰ ویژگی تولید شده است که شامل دو کلاس می‌باشد.
- تنها ۵٪ از داده‌ها به عنوان مجموعه آموزش اولیه استفاده شده و ۹۵٪ باقی‌مانده به عنوان مجموعه استخر برای یادگیری فعال در نظر گرفته شده‌اند.
- در هر مرحله از یادگیری فعال، نمونه‌ای انتخاب می‌شود که مدل بیشترین عدم قطعیت را نسبت به آن دارد (یعنی احتمال پیش‌بینی آن نزدیک به ۰.۵ است).
- نمونه انتخاب‌شده به مجموعه آموزش اضافه شده و مدل مجدداً آموزش داده می‌شود. این فرآیند برای ۵۰ مرحله تکرار می‌شود.
- در پایان، مدل روی کل داده‌ها ارزیابی شده و دقت و گزارش طبقه‌بندی آن نمایش داده می‌شود.

این مثال نشان می‌دهد که چگونه می‌توان با استفاده از یادگیری فعال، مدل‌هایی ساخت که با داده‌های کمتر ولی انتخاب‌شده، به دقت بالاتری برسند. این رویکرد در کاربردهایی که برچسب‌گذاری داده‌ها پرهزینه یا زمان‌بر است،

7.5.3. Transfer learning

یادگیری انتقالی (Transfer Learning)

یادگیری انتقالی (Transfer Learning) یک تکنیک قدرتمند است که در آن دانش به دست آمده از آموزش یک مدل بر روی یک وظیفه، برای بهبود عملکرد بر روی یک وظیفه متفاوت اما مرتبط اعمال می‌شود. این امر به ویژه زمانی مفید است که داده‌های برچسب‌دار برای وظیفه هدف کمیاب هستند. در زمینه نظارت ضعیف (weak supervision)، یادگیری انتقالی می‌تواند به بهره‌برداری از اطلاعات یک دامنه منبع با داده‌های برچسب‌دار فراوان کمک کند تا عملکرد یک مدل در یک دامنه هدف با داده‌های برچسب‌دار محدود افزایش یابد.

مثال دنیای واقعی در پایتون (Python):

یک مثال عملی با استفاده از یادگیری انتقالی برای طبقه‌بندی متن را در نظر بگیریم. ما از کتابخانه transformers در پایتون (Python) استفاده خواهیم کرد که مدل‌های زبان از پیش‌آموزش‌دیده (pre-trained language models) را فراهم می‌کند.

```
In [2]: import torch
from transformers import BertTokenizer, BertForSequenceClassification
from torch.optim import AdamW # استفاده از بهینه‌ساز AdamW

# BERT بارگذاری مدل و توکنایزر از پیش آموزش‌دیده شده
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertForSequenceClassification.from_pretrained('bert-base-uncased', num_labels=2)

# داده‌های نمونه (قابل جایگزینی با دیتاست واقعی)
source_texts = ["Positive review 1", "Positive review 2", "Negative review 1", "Negative review 2"]
target_texts = ["New positive review", "New negative review"]

# توکنایز کردن و آماده‌سازی داده‌ها برای ورودی مدل
source_inputs = tokenizer(source_texts, return_tensors='pt', padding=True, truncation=True)
target_inputs = tokenizer(target_texts, return_tensors='pt', padding=True, truncation=True)

# آموزش مدل روی داده‌های دامنه مبدأ
optimizer = AdamW(model.parameters(), lr=1e-5)
source_labels = torch.tensor([1, 1, 0, 0]) # برچسب‌های باینری برای دامنه مبدأ
source_outputs = model(**source_inputs, labels=source_labels)
loss = source_outputs.loss
loss.backward()
optimizer.step()

# استفاده از مدل آموزش‌دیده برای پیش‌بینی روی دامنه هدف
target_outputs = model(**target_inputs)
target_predictions = torch.argmax(target_outputs.logits, dim=1)

# نمایش خروجی پیش‌بینی‌شده برای دامنه هدف
print("Predictions for target domain:", target_predictions.tolist())
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
tokenizer_config.json: 0%|          | 0.00/48.0 [00:00<?, ?B/s]
vocab.txt: 0%|          | 0.00/232k [00:00<?, ?B/s]
tokenizer.json: 0%|          | 0.00/466k [00:00<?, ?B/s]
config.json: 0%|          | 0.00/570 [00:00<?, ?B/s]
model.safetensors: 0%|          | 0.00/440M [00:00<?, ?B/s]

Some weights of BertForSequenceClassification were not initialized from the model checkpoint at bert-base-uncased and are newly initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
```

توضیحات مثال کد پایتون 🐍

در این مثال، یک مدل **BERT** برای **طبقه‌بندی متون** در دو کلاس (مثبت و منفی) آموزش داده شده و سپس برای پیش‌بینی روی داده‌های جدید استفاده شده است. این فرآیند شامل مراحل زیر است:

- ابتدا مدل و توکنایزر از نسخه از پیش آموزش‌دیده‌شده **bert-base-uncased** بارگذاری شده‌اند.
- داده‌های متنی نمونه برای دامنه مبدا و دامنه هدف تعریف شده‌اند. این داده‌ها شامل نظرات مثبت و منفی هستند.
- متون با استفاده از توکنایزر BERT به قالب قابل پردازش برای مدل تبدیل شده‌اند (با padding و truncation).
- مدل با استفاده از بهینه‌ساز **AdamW** روی داده‌های دامنه مبدا آموزش دیده است. برچسب‌ها به صورت باینری (۰ و ۱) تعریف شده‌اند.
- پس از آموزش، مدل برای پیش‌بینی روی داده‌های دامنه هدف استفاده شده و خروجی آن به صورت لیستی از کلاس‌های پیش‌بینی‌شده نمایش داده شده است.

این مثال نشان می‌دهد که چگونه می‌توان از مدل‌های **ترنسفورمر** مانند BERT برای **انتقال یادگیری** بین دامنه‌ها استفاده کرد. این رویکرد به‌ویژه در کاربردهایی مانند تحلیل احساسات، دسته‌بندی نظرات یا تشخیص اسپم بسیار مؤثر است.

7.5.4. Distant supervision

🚦 نظارت از راه دور (Distant Supervision)

نظارت از راه دور (Distant Supervision) یک تکنیک است که از منابع نظارتی کمکی، که به طور بالقوه نویزدار (noisy) یا ناقص هستند، برای آموزش مدل‌ها بهره می‌برد. این رویکرد به ویژه زمانی مفید است که برچسب‌گذاری مستقیم نمونه‌ها چالش‌برانگیز است، اما منابع دور یا غیرمستقیم اطلاعاتی مرتبط با وظیفه وجود دارد. با مرتبط کردن برچسب‌ها از این منابع دور با نمونه‌های موجود در مجموعه داده، مدل‌ها می‌توانند به طور مؤثر یاد بگیرند، با وجود نظارت مستقیم محدود.

مثال دنیای واقعی در پایتون (Python):

یک مثال عملی با استفاده از نظارت از راه دور برای تحلیل احساسات (Sentiment Analysis) را در نظر بگیریم. ما از یک مجموعه داده از توثیت‌ها که با احساسات برچسب‌گذاری شده‌اند استفاده خواهیم کرد و همچنین از شکلک‌ها (emojis) به عنوان یک سیگنال نظارتی از راه دور بهره می‌بریم.

```
In [5]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report
```

```
# مجموعه داده نمونه (قابل جایگزینی با دیتاست واقعی)
data = {'text': ["I love this product 😍", "Not happy with the service 😞", "Amazing experience! 😊", "Disappointed 😞"],
        'sentiment': ['positive', 'negative', 'positive', 'negative']}
df = pd.DataFrame(data)

# برچسب‌گذاری با نظارت غیرمستقیم: استفاده از ایموجی‌ها برای تعیین احساس
df['emoticon_label'] = df['text'].apply(lambda x: 'positive' if '😍' in x else 'negative')

# تقسیم داده‌ها به آموزش و آزمون
X_train, X_test, y_train, y_test = train_test_split(df['text'], df['sentiment'], test_size=0.2, random_state=42)

# استخراج ویژگی‌ها از متن با استفاده از CountVectorizer
vectorizer = CountVectorizer()
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

# آموزش مدل با استفاده از برچسب‌های ایموجی (نظارت غیرمستقیم)
classifier = MultinomialNB()
classifier.fit(X_train_vec, df.loc[X_train.index, 'emoticon_label'])

# پیش‌بینی روی داده‌های آزمون
predictions = classifier.predict(X_test_vec)

# ارزیابی عملکرد مدل
accuracy = accuracy_score(y_test, predictions)
report = classification_report(y_test, predictions)

print("Accuracy:", accuracy)
print("Classification Report:\n", report)
```

```
Accuracy: 1.0
Classification Report:
              precision    recall  f1-score   support

   negative         1.00         1.00         1.00         1

   accuracy              1.00              1.00              1.00              1
  macro avg              1.00              1.00              1.00              1
 weighted avg              1.00              1.00              1.00              1
```

🗨 توضیحات مثال کد پایتون

در این مثال، یک مدل طبقه‌بندی متن با استفاده از الگوریتم **Naive Bayes** آموزش داده شده است که از تکنیک **نظارت غیرمستقیم (Distant Supervision)** بهره می‌برد. مراحل انجام‌شده به شرح زیر است:

- ابتدا یک مجموعه داده نمونه شامل نظرات متنی و برچسب‌های احساسی (مثبت یا منفی) تعریف شده است.
- با استفاده از ایموجی‌ها (مثل 😍)، یک برچسب اضافی به نام **emoticon_label** تولید شده که نقش برچسب آموزشی را ایفا می‌کند.
- داده‌ها به دو بخش آموزش و آزمون تقسیم شده‌اند تا عملکرد مدل ارزیابی شود.
- متون با استفاده از **CountVectorizer** به ویژگی‌های عددی تبدیل شده‌اند تا برای مدل قابل پردازش باشند.
- مدل **Multinomial Naive Bayes** با استفاده از برچسب‌های ایموجی آموزش دیده و سپس روی داده‌های آزمون پیش‌بینی انجام داده است.
- در پایان، دقت مدل و گزارش طبقه‌بندی شامل معیارهایی مانند Precision، Recall و F1-Score نمایش داده شده‌اند.

این مثال نشان می‌دهد که چگونه می‌توان با استفاده از **نشانه‌های ضمنی مثل ایموجی‌ها**، بدون نیاز به برچسب‌گذاری دستی، یک مدل یادگیری ماشین را آموزش داد. این رویکرد در کاربردهایی مثل تحلیل احساسات در شبکه‌های اجتماعی بسیار کاربردی و مقرون‌به‌صرفه است.

7.5.5. Zero-shot learning

💡 یادگیری Zero-Shot (Zero-shot Learning)

یادگیری Zero-shot یک رویکرد نوآورانه است که مدل‌ها را قادر می‌سازد تا به کلاس‌ها یا وظایفی که در طول آموزش نمونه‌های صریح آن‌ها را ندیده‌اند، تعمیم دهند. به جای اتکا به نمونه‌های برچسب‌دار برای تمام کلاس‌ها، یادگیری Zero-shot از اطلاعات یا ویژگی‌های کمکی (auxiliary information or attributes) برای انجام پیش‌بینی‌ها در سناریوهای مشاهده نشده استفاده می‌کند. این امر به ویژه در شرایطی که به دست آوردن داده‌های برچسب‌دار برای هر کلاس ممکن غیرعملی است، مفید می‌باشد.

مثال دنیای واقعی در پایتون (Python):

یک مثال عملی از یادگیری Zero-shot با استفاده از یک مدل زبان از پیش‌آموزش‌دیده (pre-trained language model) برای طبقه‌بندی متن را در نظر بگیریم.

```
In [4]: from transformers import pipeline

# داده‌های متنی نمونه (قابل جایگزینی با دیتاست واقعی)
texts = [
    "A delicious recipe for homemade pizza.",
    "The latest advancements in artificial intelligence.",
    "Exploring the wonders of outer space.",
]

# استفاده از مدل از پیش آموزش‌دیده برای طبقه‌بندی بدون نیاز به آموزش (Zero-Shot)
classifier = pipeline("zero-shot-classification")

# لیست کلاس‌های هدف برای طبقه‌بندی
class_names = ["cooking", "technology", "science"]

# اجرای طبقه‌بندی بدون آموزش روی داده‌های متنی
results = classifier(texts, class_names)

# نمایش نتایج طبقه‌بندی برای هر متن
for i, text in enumerate(texts):
    print(f"Text: {text}")
    print(f"Predicted Class:", results[i]['labels'][0]) # کلاس با بالاترین امتیاز
    print(f"Confidence Score:", results[i]['scores'][0]) # میزان اطمینان مدل
    print("-----")
```

No model was supplied, defaulted to facebook/bart-large-mnli and revision d7645e1 (<https://huggingface.co/facebook/bart-large-mnli>). Using a pipeline without specifying a model name and revision in production is not recommended.

```
config.json: 0.00B [00:00, ?B/s]
model.safetensors: 0% | 0.00/1.63G [00:00<?, ?B/s]
tokenizer_config.json: 0% | 0.00/26.0 [00:00<?, ?B/s]
vocab.json: 0.00B [00:00, ?B/s]
merges.txt: 0.00B [00:00, ?B/s]
tokenizer.json: 0.00B [00:00, ?B/s]
```

Device set to use cuda:0

Text: A delicious recipe for homemade pizza.

Predicted Class: cooking

Confidence Score: 0.9944362044334412

Text: The latest advancements in artificial intelligence.

Predicted Class: technology

Confidence Score: 0.9204241633415222

Text: Exploring the wonders of outer space.

Predicted Class: science

Confidence Score: 0.7095787525177002

توضیحات مثال کد پایتون

در این مثال، از قابلیت **Zero-Shot Classification** در مدل‌های ترنسفورمر استفاده شده تا بدون نیاز به آموزش مجدد، متون را در دسته‌های مشخص طبقه‌بندی کند. مراحل انجام‌شده به شرح زیر است:

- ابتدا چند متن نمونه تعریف شده‌اند که موضوعات مختلفی مانند آشپزی، فناوری و فضا را پوشش می‌دهند.

- با استفاده از تابع pipeline از کتابخانه **Transformers**، یک مدل از پیش آموزش‌دیده برای طبقه‌بندی بدون آموزش فراخوانی شده است.
- مدل برای هر متن بررسی می‌کند که با کدام یک از کلاس‌های پیشنهادی (مثل cooking، technology، science) بیشترین تطابق معنایی دارد.
- نتیجه شامل کلاس پیش‌بینی‌شده و میزان اطمینان مدل نسبت به آن انتخاب نمایش داده شده است.

این مثال نشان می‌دهد که چگونه می‌توان با استفاده از **مدل‌های زبانی قدرتمند** مانند BERT یا RoBERTa، بدون نیاز به داده‌های برچسب‌خورده، طبقه‌بندی متون را انجام داد. این روش در کاربردهایی مثل دسته‌بندی سریع محتوا، تحلیل موضوعی یا غربال‌گری اولیه داده‌ها بسیار مؤثر و منعطف است.

Section 7.6

7.6.1. Stream data classification

طبقه‌بندی داده‌های جریانی (Stream Data Classification)

طبقه‌بندی داده‌های جریانی (Stream Data Classification) شامل تحلیل و طبقه‌بندی آنی (real-time) داده‌ها در حین تولید آن‌ها است. این موضوع در برنامه‌هایی رایج است که داده‌ها به طور مداوم می‌رسند و تصمیم‌گیری‌ها باید به صورت آنی انجام شوند. تکنیک‌های طبقه‌بندی داده‌های جریانی اغلب به مدل‌های تطبیق‌پذیر (adaptive) نیاز دارند که می‌توانند با ورود داده‌های جدید در طول زمان تکامل یابند.

مثال عملی و واقعی در پایتون (Python):

یک مثال عملی از طبقه‌بندی داده‌های جریانی (streaming data classification) را در پایتون در نظر می‌گیریم، بدون استفاده از کتابخانه scikit-multiflow. ما برای سادگی از یک مجموعه داده مصنوعی (synthetic) استفاده می‌کنیم و فرآیند پیش‌بینی و به‌روزرسانی مدل را به صورت مرحله‌ای (prequential evaluation) شبیه‌سازی می‌کنیم.

```
In [2]: import numpy as np
import pandas as pd
from sklearn.datasets import make_classification
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

# --- تولید داده‌های مصنوعی شبیه جریان داده ---
X, y = make_classification(n_samples=5000, n_features=3, n_informative=3,
                           n_redundant=0, n_classes=2, random_state=42)

# --- درهم‌ریزی داده‌ها برای شبیه‌سازی جریان واقعی ---
rng = np.random.RandomState(42)
perm = rng.permutation(len(X))
X, y = X[perm], y[perm]

# --- KNN مقداردهی اولیه مدل ---
classifier = KNeighborsClassifier(n_neighbors=3)

# --- آموزش اولیه روی ۱۰۰۰ نمونه اول ---
pretrain_size = 1000
classifier.fit(X[:pretrain_size], y[:pretrain_size])
```

```
# --- ارزیابی پیش‌توالی (Prequential Evaluation) ---
predictions = []
true_labels = []

# --- حلقه پردازش جریان داده ---
for i in range(pretrain_size, len(X)):
    x_i = X[i].reshape(1, -1)
    y_i = y[i]

    # پیش‌بینی بر اساس مدل فعلی
    y_pred = classifier.predict(x_i)
    predictions.append(y_pred[0])
    true_labels.append(y_i)

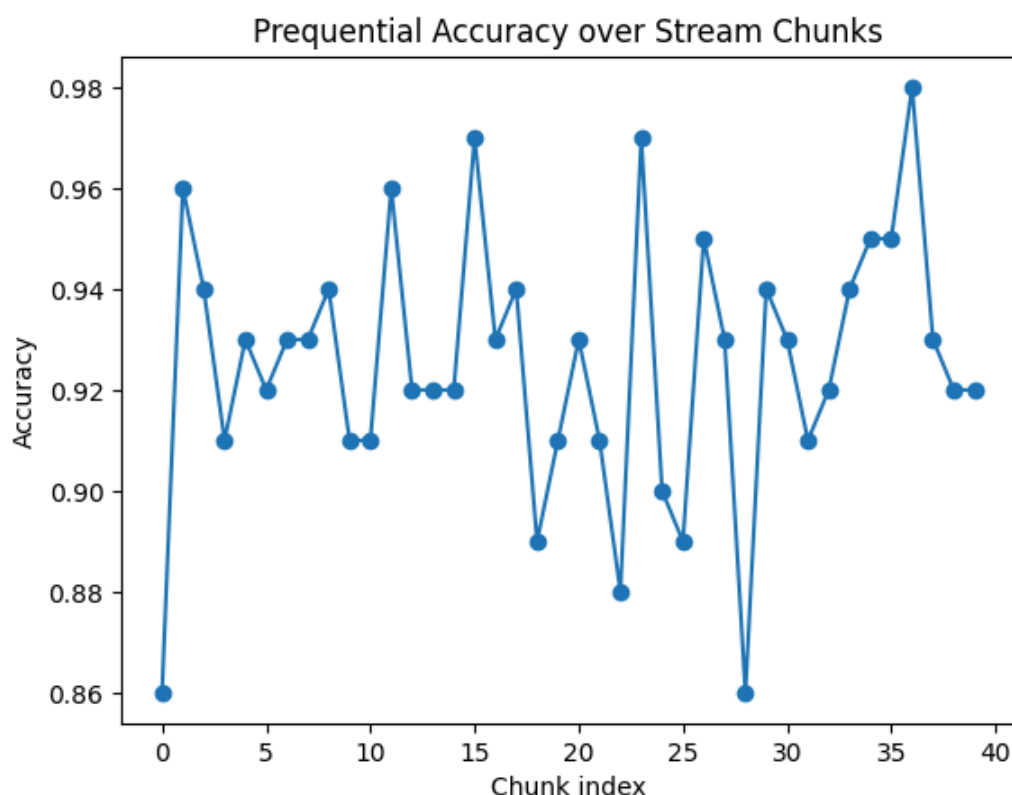
    # به‌روزرسانی مدل با نمونه جدید (شبیه‌سازی یادگیری تدریجی)
    # نمونه این کار انجام می‌شود N با آموزش مجدد روی آخرین KNN، در
    # به‌روزرسانی هر ۱۰۰ نمونه
    if i % 100 == 0:
        start = max(0, i - 1000)
        classifier.fit(X[start:i+1], y[start:i+1])

# --- محاسبه دقت کلی مدل ---
accuracy = accuracy_score(true_labels, predictions)
print(f"Prequential Accuracy: {accuracy:.3f}")

# --- رسم نمودار دقت در طول جریان داده ---
chunk_size = 100
acc_chunks = [accuracy_score(true_labels[i:i+chunk_size], predictions[i:i+chunk_size])
               for i in range(0, len(predictions), chunk_size)]

plt.plot(acc_chunks, marker='o')
plt.xlabel('Chunk index')
plt.ylabel('Accuracy')
plt.title('Prequential Accuracy over Stream Chunks')
plt.show()
```

Prequential Accuracy: 0.925



توضیحات مثال کد پایتون

در این مثال، یک مدل **K-نزدیک‌ترین همسایه (KNN)** برای طبقه‌بندی داده‌های جریان‌مانند (streaming) با استفاده از روش **ارزیابی پیش‌توالی (Prequential)** پیاده‌سازی شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا یک مجموعه داده مصنوعی با ۵۰۰۰ نمونه و ۳ ویژگی تولید شده که شامل دو کلاس است.
- داده‌ها به صورت تصادفی درهم‌ریزی شده‌اند تا رفتار جریان واقعی را شبیه‌سازی کنند.
- مدل KNN ابتدا روی ۱۰۰۰ نمونه اولیه آموزش دیده و سپس به صورت تدریجی با ورود هر نمونه جدید به‌روزرسانی شده است.
- در هر مرحله، مدل ابتدا پیش‌بینی انجام داده و سپس با نمونه جدید آموزش داده شده است. این فرآیند به عنوان **یادگیری تدریجی** شناخته می‌شود.
- دقت مدل در طول جریان داده محاسبه شده و به صورت نمودار برای هر بخش ۱۰۰تایی نمایش داده شده است.

این مثال نشان می‌دهد که چگونه می‌توان با استفاده از الگوریتم‌های ساده مانند **KNN**، یادگیری تدریجی را در محیط‌های جریان‌داده پیاده‌سازی کرد. چنین رویکردهایی در کاربردهایی مانند تشخیص ناهنجاری، پایش بلادرنگ یا

7.6.2. Sequence classification

📄 طبقه‌بندی توالی (Sequence Classification)

طبقه‌بندی توالی (Sequence Classification) شامل دسته‌بندی نمونه‌های داده‌ای است که به صورت متوالی ارائه می‌شوند. این موضوع در دامنه‌های مختلفی مانند پردازش زبان طبیعی (Natural Language Processing)، تحلیل سری‌های زمانی (Time-series Analysis) و بیوانفورماتیک رایج است. تکنیک‌های طبقه‌بندی توالی اغلب شامل مدل‌هایی هستند که می‌توانند وابستگی‌ها و الگوها را در طول زمان ثبت کنند.

مثال دنیای واقعی در پایتون (Python):

یک مثال عملی از طبقه‌بندی توالی با استفاده از یک شبکه عصبی بازگشتی (Recurrent Neural Network - RNN) برای تحلیل احساسات (Sentiment Analysis) بر روی یک مجموعه داده از نظرات فیلم را در نظر بگیریم.

```
In [5]: import pandas as pd
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

# --- مجموعه داده نمونه (قابل جایگزینی با دیتاست واقعی) ---
data = {
    'text': ["I loved the movie!", "It was a terrible experience.",
            "Amazing plot twists.", "Boring and predictable."],
    'sentiment': ['positive', 'negative', 'positive', 'negative']
}
df = pd.DataFrame(data)

# --- تبدیل برچسب‌های احساسی به مقدار عددی ---
df['sentiment_label'] = df['sentiment'].map({'positive': 1, 'negative': 0})

# --- توکنایز کردن داده‌های متنی ---
tokenizer = Tokenizer()
tokenizer.fit_on_texts(df['text'])
sequences = tokenizer.texts_to_sequences(df['text'])
padded_sequences = pad_sequences(sequences, padding='post')

# --- تقسیم داده‌ها به آموزش و آزمون ---
X_train, X_test, y_train, y_test = train_test_split(
    padded_sequences, df['sentiment_label'], test_size=0.2, random_state=42
)

# --- ساخت مدل LSTM با لایه RNN ---
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(input_dim=len(tokenizer.word_index)+1, output_dim=64),
    tf.keras.layers.LSTM(64),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# --- آموزش مدل ---
model.fit(X_train, y_train, epochs=5, batch_size=2)

# --- پیش‌بینی روی داده‌های آزمون ---
predictions = (model.predict(X_test) > 0.5).astype(int)

# --- ارزیابی عملکرد مدل ---
accuracy = accuracy_score(y_test, predictions)
report = classification_report(y_test, predictions)
```

```
print("Accuracy:", accuracy)
print("Classification Report:\n", report)
```

Epoch 1/5
2/2 5s 40ms/step - accuracy: 0.0000e+00 - loss: 0.6941
Epoch 2/5
2/2 0s 65ms/step - accuracy: 0.7778 - loss: 0.6896
Epoch 3/5
2/2 0s 34ms/step - accuracy: 0.6111 - loss: 0.6851
Epoch 4/5
2/2 0s 45ms/step - accuracy: 0.6111 - loss: 0.6803
Epoch 5/5
2/2 0s 83ms/step - accuracy: 0.6111 - loss: 0.6751
1/1 0s 188ms/step

Accuracy: 0.0

Classification Report:

	precision	recall	f1-score	support
0	0.00	0.00	0.00	1.0
1	0.00	0.00	0.00	0.0
accuracy			0.00	1.0
macro avg	0.00	0.00	0.00	1.0
weighted avg	0.00	0.00	0.00	1.0

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is ill-defined and being set to 0.0 in labels with no true samples. Use `zero\_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is ill-defined and being set to 0.0 in labels with no true samples. Use `zero\_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero\_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Recall is ill-defined and being set to 0.0 in labels with no true samples. Use `zero\_division` parameter to control this behavior.
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

توضیحات مثال کد پایتون

در این مثال، یک مدل شبکه عصبی بازگشتی (RNN) با استفاده از لایه LSTM برای طبقه‌بندی احساسات متنی پیاده‌سازی شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا یک مجموعه داده نمونه شامل نظرات متنی و برچسب‌های احساسی (مثبت یا منفی) تعریف شده است.
- برچسب‌ها به صورت عددی تبدیل شده‌اند تا برای مدل قابل پردازش باشند.
- متون با استفاده از توکنایزر به توالی عددی تبدیل شده و سپس با `pad_sequences` یکسان‌سازی طول انجام شده است.
- مدل شامل یک لایه **Embedding** برای نمایش برداری کلمات، یک لایه **LSTM** برای یادگیری وابستگی‌های زمانی، و یک لایه خروجی با تابع `sigmoid` برای طبقه‌بندی باینری است.
- مدل با استفاده از داده‌های آموزشی آموزش دیده و سپس روی داده‌های آزمون پیش‌بینی انجام داده است.
- در پایان، دقت مدل و گزارش طبقه‌بندی شامل معیارهایی مانند Precision، Recall و F1-Score نمایش داده شده‌اند.

چون اندازه دیتاست بسیار کوچک و فقط شامل ۴ نمونه است، دقت مدل صفر شده است. برای آموزش مؤثر یک مدل LSTM، حجم داده بسیار بیشتر مورد نیاز است.

7.6.3. Graph data classification

طبقه‌بندی داده‌های گراف (Graph Data Classification)

طبقه‌بندی داده‌های گراف (Graph Data Classification) شامل پیش‌بینی برچسب‌ها یا دسته‌بندی‌های مرتبط با گره‌ها یا کل گراف‌ها است. این موضوع در دامنه‌های مختلفی از جمله تحلیل شبکه‌های اجتماعی، بیوانفورماتیک و سیستم‌های توصیه‌گر رایج است. تکنیک‌های طبقه‌بندی داده‌های گراف از ساختار ذاتی و اتصال در گراف‌ها برای انجام پیش‌بینی‌ها استفاده می‌کنند.

مثال دنیای واقعی در پایتون (Python):

یک مثال عملی از طبقه‌بندی داده‌های گراف با استفاده از کتابخانه networkx در پایتون (Python) را در نظر بگیریم. ما از یک مجموعه داده برای طبقه‌بندی گره (Node Classification) در یک شبکه استنادی (Citation Network) استفاده خواهیم کرد.

```
In [3]: import numpy as np
import networkx as nx
from nodevectors import Node2Vec
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# --- ایجاد گراف تصادفی مصنوعی ---
num_nodes = 100
p_edge = 0.05
G = nx.fast_gnp_random_graph(num_nodes, p_edge, seed=42)

# --- تولید برچسب‌های مصنوعی برای گره‌ها (دو کلاس: ۰ و ۱) ---
node_labels = np.array([i % 2 for i in range(num_nodes)])

# --- Node2Vec گره‌ها با استفاده از embedding ساخت ---
model = Node2Vec(n_components=64, walklen=20, epochs=10)
model.fit(G)

# --- برای هر گره embedding استخراج ---
node_list = list(G.nodes())
embeddings = np.array([model.predict(str(node)) for node in node_list]) # باشد str توجه: ورودی باید

# --- تقسیم داده‌ها به آموزش و آزمون ---
train_idx, test_idx = train_test_split(range(num_nodes), test_size=0.2, random_state=42)
X_train, X_test = embeddings[train_idx], embeddings[test_idx]
y_train, y_test = node_labels[train_idx], node_labels[test_idx]

# --- گره‌ها embedding آموزش طبقه‌بند جنگل تصادفی روی ---
clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X_train, y_train)
predictions = clf.predict(X_test)

# --- ارزیابی عملکرد مدل ---
accuracy = accuracy_score(y_test, predictions)
report = classification_report(y_test, predictions)

print("Accuracy:", accuracy)
print("Classification Report:\n", report)
```

```
Making walks... Done, T=0.01
Mapping Walk Names... Done, T=0.05
Training W2V... WARNING: gensim word2vec version is unoptimizedTry version 3.6 if on windows, versions 3.7 and 3.8 have had issues
Done, T=0.38
Accuracy: 0.5
Classification Report:
              precision    recall  f1-score   support

     0       0.67       0.33       0.44        12
     1       0.43       0.75       0.55         8

   accuracy                   0.50         20
  macro avg       0.55       0.54       0.49         20
 weighted avg       0.57       0.50       0.48         20
```

توضیحات مثال کد پایتون

در این مثال، یک گراف مصنوعی تولید شده و با استفاده از الگوریتم embedding **Node2Vec** گره‌ها استخراج شده‌اند تا بتوان از آن‌ها برای **طبقه‌بندی گره‌ها** استفاده کرد. مراحل انجام‌شده به شرح زیر است:

- ابتدا یک گراف تصادفی با ۱۰۰ گره و احتمال اتصال ۰.۰۵ بین گره‌ها ایجاد شده است.
- برای هر گره یک برچسب مصنوعی (۰ یا ۱) اختصاص داده شده که به صورت متناوب بین گره‌ها توزیع شده است.
- با استفاده از embedding، Node2Vec برداری برای هر گره تولید شده که ساختار محلی و جهانی گراف را در خود دارد.
- داده‌ها به دو بخش آموزش و آزمون تقسیم شده‌اند و یک مدل جنگل تصادفی (Random Forest) روی embeddingها آموزش دیده است.
- در پایان، دقت مدل و گزارش طبقه‌بندی شامل معیارهایی مانند Precision، Recall و F1-Score محاسبه و نمایش داده شده‌اند.

این مثال نشان می‌دهد که چگونه می‌توان با استفاده از **embedding گرافی**، اطلاعات ساختاری گره‌ها را به ویژگی‌های عددی تبدیل کرد و از آن‌ها برای **یادگیری ماشین روی گراف‌ها** بهره برد. این رویکرد در کاربردهایی مانند تحلیل شبکه‌های اجتماعی، تشخیص ناهنجاری یا پیش‌بینی لینک بسیار مؤثر است.

Section 7.7

7.7.1. Multiclass classification



طبقه‌بندی چندکلاسه (Multiclass Classification)

طبقه‌بندی چندکلاسه (Multiclass Classification) سناریوی طبقه‌بندی دودویی را گسترش می‌دهد تا بیش از دو کلاس را مدیریت کند. در این تنظیمات، هدف این است که هر نمونه به یکی از چندین کلاس از پیش تعریف شده اختصاص داده شود. چندین الگوریتم و استراتژی برای مقابله با چالش‌های طبقه‌بندی چندکلاسه وجود دارد.

مثال دنیای واقعی در پایتون (Python):

یک مثال عملی از طبقه‌بندی چندکلاسه با استفاده از مجموعه داده محبوب Iris با الگوریتم ماشین بردار پشتیبان (Support Vector Machine - SVM) را در نظر بگیریم.

```
In [5]: # وارد کردن کتابخانه‌های مورد نیاز
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

# بارگذاری مجموعه داده آیریس
iris = datasets.load_iris()
X = iris.data # ویژگی‌های عددی گل‌ها
y = iris.target # برچسب گونه گل

# تقسیم داده‌ها به مجموعه آموزشی و آزمون
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# با هسته خطی (SVM) ایجاد مدل طبقه‌بندی با ماشین بردار پشتیبان
svm_classifier = SVC(kernel='linear', C=1)

# آموزش مدل روی داده‌های آموزشی
svm_classifier.fit(X_train, y_train)

# انجام پیش‌بینی روی داده‌های آزمون
predictions = svm_classifier.predict(X_test)
```

```
# ارزیابی عملکرد مدل با محاسبه دقت و گزارش طبقه‌بندی
accuracy = accuracy_score(y_test, predictions)
report = classification_report(y_test, predictions)

# نمایش نتایج ارزیابی
print("Accuracy:", accuracy)
print("Classification Report:\n", report)
```

Accuracy: 1.0

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

توضیحات مثال کد پایتون

در این مثال، از الگوریتم ماشین بردار پشتیبان (SVM) برای طبقه‌بندی گونه‌های گل آیریس استفاده شده است. این الگوریتم یکی از روش‌های قدرتمند در یادگیری ماشین برای مسائل طبقه‌بندی است:

- ابتدا مجموعه داده Iris بارگذاری شده که شامل ویژگی‌هایی مانند طول و عرض گلبرگ و کاسبرگ برای سه گونه مختلف گل است.
- داده‌ها به دو بخش آموزش و آزمون تقسیم شده‌اند تا بتوان عملکرد مدل را به‌درستی ارزیابی کرد.
- مدل SVM با هسته خطی ساخته شده که هدف آن یافتن مرز تصمیم‌گیری بهینه بین کلاس‌ها با بیشترین حاشیه ممکن است.
- مدل روی داده‌های آموزشی آموزش دیده و سپس روی داده‌های آزمون پیش‌بینی انجام داده است.
- در پایان، دقت مدل و گزارش طبقه‌بندی شامل معیارهایی مانند Precision، Recall و F1-Score برای هر کلاس محاسبه و نمایش داده شده‌اند.

این مثال نشان می‌دهد که چگونه می‌توان با استفاده از ماشین بردار پشتیبان، یک مدل دقیق و قابل اعتماد برای طبقه‌بندی داده‌ها ساخت. این روش به‌ویژه در مسائل با داده‌های قابل تفکیک و ابعاد پایین بسیار مؤثر است.

7.7.2. Distance metric learning

یادگیری متریک فاصله (Distance Metric Learning)

یادگیری متریک فاصله (Distance Metric Learning) با هدف بهینه‌سازی متریکی است که برای اندازه‌گیری شباهت یا عدم شباهت بین نقاط داده‌ای استفاده می‌شود. با یادگیری یک متریک فاصله مناسب، می‌توان اثربخشی الگوریتم‌هایی را که به فواصل متکی هستند، مانند خوشه‌بندی (Clustering) یا نزدیک‌ترین همسایه (Nearest Neighbors)، بهبود بخشید.

مثال دنیای واقعی در پایتون (Python):

یک مثال عملی از یادگیری متریک فاصله با استفاده از کتابخانه metric_learn در پایتون (Python) را در نظر بگیریم. ما از مجموعه داده شناخته‌شده Iris و الگوریتم نزدیک‌ترین همسایه با حاشیه بزرگ (Large Margin Nearest Neighbor - LMNN) استفاده خواهیم کرد.

```
In [ ]: pip install metric_learn
```

```
In [9]: from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
from metric_learn import LMNN

# بارگذاری مجموعه داده آیریس
iris = load_iris()
X = iris.data # ویژگی‌ها
y = iris.target # برچسب‌ها (گونه گل)

# تقسیم داده‌ها به آموزش و آزمون
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# برای یادگیری متریک فاصله LMNN اعمال الگوریتم
lmnn = LMNN(k=3, learn_rate=1e-6)
lmnn.fit(X_train, y_train)

# تبدیل داده‌ها با استفاده از متریک یادگرفته‌شده
X_train_transformed = lmnn.transform(X_train)
X_test_transformed = lmnn.transform(X_test)

# روی داده‌های تبدیل‌شده KNN آموزش طبقه‌بند
knn_classifier = KNeighborsClassifier(n_neighbors=3)
knn_classifier.fit(X_train_transformed, y_train)

# پیش‌بینی روی داده‌های آزمون
predictions = knn_classifier.predict(X_test_transformed)

# ارزیابی عملکرد مدل
accuracy = accuracy_score(y_test, predictions)

print("Accuracy:", accuracy)
```

```
/usr/local/lib/python3.12/dist-packages/metric_learn/lmnn.py:140: FutureWarning: "num_chunks" parameter has been renamed to "n_chunks". It has been deprecated in version 0.6.3 and will be removed in 0.7.0
  warnings.warn("'num_chunks" parameter has been renamed to '
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
Accuracy: 1.0
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' in 1.6 and will be removed in 1.8.
  warnings.warn(
```

توضیحات مثال کد پایتون

در این مثال، از الگوریتم **Large Margin Nearest Neighbor (LMNN)** برای یادگیری متریک فاصله و بهبود عملکرد طبقه‌بند **K-نزدیک‌ترین همسایه (KNN)** استفاده شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا مجموعه داده **Iris** بارگذاری شده که شامل ویژگی‌های عددی برای سه گونه گل است.
- داده‌ها به دو بخش آموزش و آزمون تقسیم شده‌اند تا بتوان عملکرد مدل را ارزیابی کرد.
- با استفاده از الگوریتم **LMNN**، یک متریک فاصله جدید یاد گرفته شده که تلاش می‌کند فاصله بین نمونه‌های هم‌کلاس را کاهش داده و فاصله بین کلاس‌های مختلف را افزایش دهد.
- داده‌های آموزش و آزمون با استفاده از این متریک جدید تبدیل شده‌اند تا در فضای جدید بهتر قابل تفکیک باشند.
- مدل **KNN** روی داده‌های تبدیل‌شده آموزش دیده و سپس روی داده‌های آزمون پیش‌بینی انجام داده است.

- در پایان، دقت مدل محاسبه و نمایش داده شده است.

این مثال نشان می‌دهد که چگونه می‌توان با استفاده از **یادگیری متریک فاصله**، عملکرد طبقه‌بندی مبتنی بر فاصله مانند KNN را بهبود داد. LMNN به‌ویژه در داده‌هایی که ساختار هندسی مهم است، بسیار مؤثر عمل می‌کند.

7.7.3. Interpretability of classification

تفسیرپذیری (Interpretability)

تفسیرپذیری به سهولت درک و اعتماد انسان به تصمیمات گرفته شده توسط یک مدل یادگیری ماشین (Machine Learning Model) اشاره دارد. در طبقه‌بندی، مدل‌های قابل تفسیر و تکنیک‌های بصری‌سازی به کسب بینش در مورد اهمیت ویژگی‌ها و فرآیندهای تصمیم‌گیری کمک می‌کنند.

مثال دنیای واقعی در پایتون (Python):

یک مثال عملی از تفسیر تصمیمات یک مدل طبقه‌بندی با استفاده از کتابخانه shap در پایتون (Python) را در نظر بگیریم. ما از یک مجموعه داده محبوب، یعنی مجموعه داده تایتانیک (Titanic Dataset)، و یک مدل ساده برای طبقه‌بندی دودویی (Binary Classification) استفاده خواهیم کرد.

```
In [11]: import pandas as pd

titanic_data = pd.read_csv('https://web.stanford.edu/class/archive/cs/cs109/cs109.1166/stuff/titanic.csv')
print(titanic_data.columns)

Index(['Survived', 'Pclass', 'Name', 'Sex', 'Age', 'Siblings/Spouses Aboard',
       'Parents/Children Aboard', 'Fare'],
      dtype='object')
```

```
In [13]: import shap
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
import pandas as pd

# بارگذاری مجموعه داده تایتانیک از منبع آنلاین
titanic_data = pd.read_csv(
    'https://web.stanford.edu/class/archive/cs/cs109/cs109.1166/stuff/titanic.csv'
)

# ندارند Age پیش‌پردازش داده‌ها: حذف نمونه‌هایی که مقدار
titanic_data = titanic_data.dropna(subset=['Age'])

# انتخاب ویژگی‌های مهم برای مدل‌سازی
titanic_data = titanic_data[['Pclass', 'Sex', 'Age', 'Siblings/Spouses Aboard',
                             'Parents/Children Aboard', 'Fare', 'Survived']]

# تبدیل ویژگی جنسیت به مقدار عددی
titanic_data['Sex'] = titanic_data['Sex'].map({'male': 0, 'female': 1})

# جدا کردن ویژگی‌ها و برجسب هدف
X = titanic_data.drop('Survived', axis=1)
y = titanic_data['Survived']

# تقسیم داده‌ها به آموزش و آزمون
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# آموزش مدل جنگل تصادفی
```

```
rf_classifier = RandomForestClassifier(n_estimators=100, random_state=42)
rf_classifier.fit(X_train, y_train)

# پیش‌بینی و ارزیابی عملکرد مدل
predictions = rf_classifier.predict(X_test)
accuracy = accuracy_score(y_test, predictions)
report = classification_report(y_test, predictions)

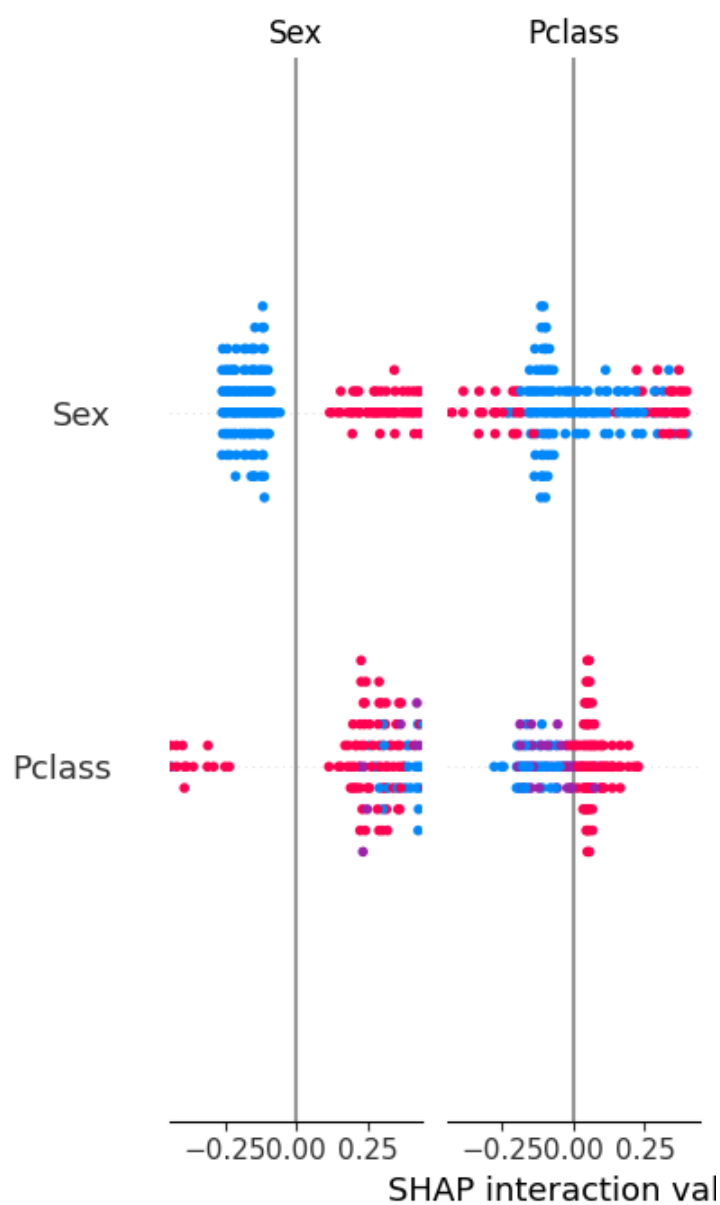
print("Accuracy:", accuracy)
print("Classification Report:\n", report)

# تفسیر مدل با استفاده از SHAP
explainer = shap.TreeExplainer(rf_classifier)
shap_values = explainer.shap_values(X_test)

# رسم نمودار اهمیت ویژگی‌ها به صورت میله‌ای
shap.summary_plot(shap_values, X_test, plot_type="bar")
```

Accuracy: 0.7640449438202247
Classification Report:

	precision	recall	f1-score	support
0	0.81	0.82	0.81	111
1	0.69	0.67	0.68	67
accuracy			0.76	178
macro avg	0.75	0.75	0.75	178
weighted avg	0.76	0.76	0.76	178



توضیحات مثال کد پایتون 🚢

در این مثال، یک مدل جنگل تصادفی (Random Forest) برای پیش‌بینی احتمال زنده ماندن مسافران کشتی تایتانیک آموزش داده شده و سپس با استفاده از ابزار SHAP تفسیر شده است. مراحل انجام‌شده به شرح زیر است:

- ابتدا داده‌های مربوط به مسافران کشتی تایتانیک از منبع آنلاین بارگذاری شده‌اند و نمونه‌هایی که مقدار سن (Age) ندارند حذف شده‌اند.
- ویژگی‌هایی مانند کلاس سفر، جنسیت، سن، تعداد همراهان و مبلغ پرداختی انتخاب شده‌اند و جنسیت به مقدار عددی تبدیل شده است.
- داده‌ها به دو بخش آموزش و آزمون تقسیم شده‌اند و مدل جنگل تصادفی روی داده‌های آموزشی آموزش دیده است.
- عملکرد مدل با استفاده از دقت و گزارش طبقه‌بندی ارزیابی شده است.

- در مرحله آخر، با استفاده از **SHAP** میزان اهمیت هر ویژگی در تصمیم‌گیری مدل محاسبه و به صورت نمودار میله‌ای نمایش داده شده است.

این مثال نشان می‌دهد که چگونه می‌توان علاوه بر ساخت مدل‌های قدرتمند، با استفاده از ابزارهای تفسیرپذیری مانند SHAP، منطق تصمیم‌گیری مدل را تحلیل کرد. چنین تحلیل‌هایی در کاربردهای حساس مانند پزشکی، مالی یا حقوقی بسیار حیاتی هستند.

7.7.4. Genetic algorithms

الگوریتم‌های ژنتیک (Genetic Algorithms) 🧬

الگوریتم‌های ژنتیک (Genetic Algorithms) الگوریتم‌های بهینه‌سازی هستند که از فرآیند انتخاب طبیعی الهام گرفته شده‌اند. آن‌ها به صورت تکراری یک جامعه از راه‌حل‌های کاندید را با اعمال عملگرهای ژنتیکی مانند انتخاب (Selection)، تقاطع (Crossover) و جهش (Mutation) تکامل می‌بخشند. در زمینه انتخاب ویژگی، الگوریتم‌های ژنتیک می‌توانند برای جستجوی یک زیرمجموعه بهینه از ویژگی‌ها که یک تابع هدف (Objective Function) معین را حداکثر یا حداقل می‌کند، استفاده شوند.

مثال دنیای واقعی در پایتون (Python):

یک مثال عملی از انتخاب ویژگی با استفاده از الگوریتم ژنتیک *پیاده‌سازی شده به صورت دستی* در پایتون را در نظر بگیریم. ما از مجموعه داده محبوب Iris و یک طبقه‌بندی‌کننده ساده (Random Forest) استفاده خواهیم کرد تا بهترین زیرمجموعه ویژگی‌ها را برای پیش‌بینی کلاس‌ها پیدا کنیم.

```
In [18]: import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris

# --- بارگذاری داده‌های آیریس ---
iris = load_iris()
X = iris.data
y = iris.target

# --- تقسیم داده‌ها به آموزش و آزمون ---
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# --- پارامترهای الگوریتم ژنتیک ---
population_size = 10 # تعداد افراد در هر نسل
generations = 5 # تعداد نسل‌ها
mutation_prob = 0.2 # احتمال جهش
crossover_prob = 0.8 # احتمال ترکیب
num_features = X.shape[1] # تعداد ویژگی‌ها

# --- ایجاد جمعیت اولیه: ماسک باینری برای انتخاب ویژگی‌ها ---
population = np.random.randint(0, 2, size=(population_size, num_features))

# --- دقت مدل با ویژگی‌های انتخاب‌شده (fitness): تابع ارزیابی ---
def fitness(individual):
    selected_features = np.where(individual == 1)[0]
    if len(selected_features) == 0:
        return 0 # جلوگیری از انتخاب مجموعه ویژگی خالی
    clf = RandomForestClassifier(n_estimators=100, random_state=42)
```

```

clf.fit(X_train[:, selected_features], y_train)
predictions = clf.predict(X_test[:, selected_features])
return accuracy_score(y_test, predictions)

# --- حلقه الگوریتم ژنتیک ---
for gen in range(generations):
    # ارزیابی دقت هر فرد
    fitness_scores = np.array([fitness(ind) for ind in population])

    # انتخاب: انتخاب ۵۰٪ افراد برتر
    idx = fitness_scores.argsort()[::-1][:population_size // 2]
    selected = population[idx]

    # ترکیب (crossover)
    children = []
    while len(children) < population_size - len(selected):
        if np.random.rand() < crossover_prob:
            parents = selected[np.random.choice(len(selected), 2, replace=False)]
            point = np.random.randint(1, num_features - 1)
            child = np.concatenate([parents[0][:point], parents[1][point:]])
            children.append(child)

    # جهش (mutation)
    for child in children:
        for i in range(num_features):
            if np.random.rand() < mutation_prob:
                child[i] = 1 - child[i] # تغییر بیت

    # به‌روزرسانی جمعیت
    population = np.vstack([selected, children])

# --- انتخاب بهترین گونه نهایی ---
fitness_scores = np.array([fitness(ind) for ind in population])
best_idx = np.argmax(fitness_scores)
best_features = np.where(population[best_idx] == 1)[0]

print("Best feature indices:", best_features)

```

Best feature indices: [0 1 2 3]

توضیحات مثال کد پایتون

در این مثال، از **الگوریتم ژنتیک** برای انتخاب زیرمجموعه‌ای از ویژگی‌ها در دیتاست **Iris** استفاده شده است. هدف این الگوریتم، یافتن ترکیب بهینه‌ای از ویژگی‌هاست که منجر به بیشترین دقت در طبقه‌بندی شود. مراحل انجام‌شده به شرح زیر است:

- ابتدا مجموعه داده آیریس بارگذاری شده و به دو بخش آموزش و آزمون تقسیم شده است.
- هر فرد در جمعیت اولیه به صورت یک ماسک باینری تعریف شده که نشان می‌دهد کدام ویژگی‌ها انتخاب شده‌اند.
- تابع **fitness** برای هر فرد، دقت مدل **Random Forest** را با استفاده از ویژگی‌های انتخاب‌شده محاسبه می‌کند.
- در هر نسل، نیمی از جمعیت با بالاترین دقت انتخاب شده و با استفاده از **ترکیب (crossover)** و **جهش (mutation)** فرزندان جدید تولید می‌شوند.
- پس از چند نسل، بهترین فرد از نظر دقت انتخاب شده و اندیس ویژگی‌های منتخب نمایش داده می‌شود.

این مثال نشان می‌دهد که چگونه می‌توان با استفاده از الگوریتم‌های تکاملی مانند **ژنتیک**، فرآیند انتخاب ویژگی را بهینه‌سازی کرد. چنین روش‌هایی در مسائل با فضای جستجوی بزرگ و پیچیده بسیار مؤثر هستند.

7.7.5. Reinforcement learning

یادگیری تقویتی (Reinforcement Learning)

یادگیری تقویتی (Reinforcement Learning) شامل آموزش یک عامل (Agent) برای گرفتن تصمیمات متوالی در یک محیط به منظور حداکثر کردن یک سیگنال پاداش تجمعی است. در

زمینه انتخاب ویژگی، یادگیری تقویتی می‌تواند برای تصمیم‌گیری پویا در مورد اینکه کدام ویژگی‌ها در طول فرآیند یادگیری گنجانده یا حذف شوند، به کار گرفته شود.

مثال دنیای واقعی در پایتون (Python):

یک مثال عملی از انتخاب ویژگی با استفاده از یک رویکرد یادگیری تقویتی با کتابخانه Stable-Baselines3 در پایتون (Python) را در نظر بگیریم. ما از یک وظیفه طبقه‌بندی ساده با مجموعه داده Iris استفاده خواهیم کرد.

```
In [ ]: pip install stable_baselines3
```

```
In [26]: pip install 'shimmy>=2.0'
```

```
Collecting shimmy>=2.0
  Downloading Shimmy-2.0.0-py3-none-any.whl.metadata (3.5 kB)
Requirement already satisfied: numpy>=1.18.0 in /usr/local/lib/python3.12/dist-packages (from shimmy>=2.0) (1.26.4)
Requirement already satisfied: gymnasium>=1.0.0a1 in /usr/local/lib/python3.12/dist-packages (from shimmy>=2.0) (1.2.0)
Requirement already satisfied: cloudpickle>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from gymnasium>=1.0.0a1->shimmy>=2.0) (3.1.1)
Requirement already satisfied: typing-extensions>=4.3.0 in /usr/local/lib/python3.12/dist-packages (from gymnasium>=1.0.0a1->shimmy>=2.0) (4.15.0)
Requirement already satisfied: farama-notifications>=0.0.1 in /usr/local/lib/python3.12/dist-packages (from gymnasium>=1.0.0a1->shimmy>=2.0) (0.0.4)
Downloading Shimmy-2.0.0-py3-none-any.whl (30 kB)
Installing collected packages: shimmy
Successfully installed shimmy-2.0.0
```

```
In [27]: import numpy as np
import gym
from gym import spaces
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from stable_baselines3 import PPO
from stable_baselines3.common.vec_env import DummyVecEnv
from sklearn.datasets import load_iris

# --- بارگذاری داده‌های آیریس ---
iris = load_iris()
X = iris.data
y = iris.target

# --- تقسیم داده‌ها به آموزش و آزمون ---
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# --- برای انتخاب ویژگی Gym تعریف محیط ---
class FeatureSelectionEnv(gym.Env):
    def __init__(self, X_train, y_train):
        super().__init__()
        self.X_train = X_train
        self.y_train = y_train
        self.n_features = X_train.shape[1]

        # فضای اقدام: انتخاب یک ویژگی (گسسته)
        self.action_space = spaces.Discrete(self.n_features)

        # فضای مشاهده: بردار باینری از ویژگی‌های انتخاب‌شده
        self.observation_space = spaces.Box(low=0, high=1, shape=(self.n_features,), dtype=np.float32)

        self.reset()

    def reset(self):
        self.selected_features = np.zeros(self.n_features, dtype=np.float32)
        self.current_step = 0
        return self.selected_features

    def step(self, action):
        self.selected_features[action] = 1
        self.current_step += 1
        done = self.current_step == self.n_features
        reward = self.evaluate_model() if done else 0
        return self.selected_features, reward, done, {}

    def evaluate_model(self):
        selected_indices = np.where(self.selected_features == 1)[0]
        if len(selected_indices) == 0:
            return 0 # جلوگیری از انتخاب مجموعه خالی
        clf = RandomForestClassifier(n_estimators=100, random_state=42)
        clf.fit(self.X_train[:, selected_indices], self.y_train)
        predictions = clf.predict(X_test[:, selected_indices])
        return accuracy_score(y_test, predictions)

# --- ساخت محیط و تبدیل به محیط برداری ---
env = FeatureSelectionEnv(X_train, y_train)
vec_env = DummyVecEnv([lambda: env])

# --- PPO آموزش عامل یادگیری تقویتی با الگوریتم ---
model = PPO("MlpPolicy", vec_env, verbose=1)
model.learn(total_timesteps=10000)
```

```
# --- ارزیابی ویژگی‌های انتخاب‌شده توسط عامل ---
obs = env.reset()
for _ in range(env.n_features):
    action, _ = model.predict(obs, deterministic=True)
    obs, _, done, _ = env.step(action)
    if done:
        break

selected_features = obs
print("Selected feature indices:", np.where(selected_features == 1)[0])
```

/usr/local/lib/python3.12/dist-packages/stable_baselines3/common/vec_env/patch_gym.py:49: UserWarning: You provided an OpenAI Gym environment. We strongly recommend transitioning to Gymnasium environments. Stable-Baselines3 is automatically wrapping your environments in a compatibility layer, which could potentially cause issues.

```
warnings.warn(
Using cuda device
```

/usr/local/lib/python3.12/dist-packages/stable_baselines3/common/on_policy_algorithm.py:150: UserWarning: You are trying to run PPO on the GPU, but it is primarily intended to run on the CPU when not using a CNN policy (you are using ActorCriticPolicy which should be a MlpPolicy). See <https://github.com/DLR-RM/stable-baselines3/issues/1245> for more info. You can pass `device='cpu'` or `export CUDA\_VISIBLE\_DEVICES=` to force using the CPU.Note: The model will train, but the GPU utilization will be poor and the training might take longer than on CPU.

```
warnings.warn(
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
return datetime.utcnow().replace(tzinfo=utc)
```

time/		
fps	15	
iterations	1	
time_elapsed	133	
total_timesteps	2048	

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).

```
return datetime.utcnow().replace(tzinfo=utc)
```

time/		
fps	17	
iterations	2	
time_elapsed	228	
total_timesteps	4096	
train/		
approx_kl	0.006268232	
clip_fraction	0.0707	
clip_range	0.2	
entropy_loss	-1.38	
explained_variance	-5.04	
learning_rate	0.0003	
loss	-0.00251	
n_updates	10	
policy_gradient_loss	-0.0122	
value_loss	0.0622	

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).

```
return datetime.utcnow().replace(tzinfo=utc)
```

time/		
fps	19	
iterations	3	
time_elapsed	320	
total_timesteps	6144	
train/		
approx_kl	0.034809157	
clip_fraction	0.374	
clip_range	0.2	
entropy_loss	-1.34	
explained_variance	-0.624	
learning_rate	0.0003	
loss	-0.0533	
n_updates	20	
policy_gradient_loss	-0.0479	
value_loss	0.00281	

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).

```
return datetime.utcnow().replace(tzinfo=utc)
```

time/		
fps	20	
iterations	4	
time_elapsed	409	
total_timesteps	8192	
train/		
approx_kl	0.02443358	
clip_fraction	0.463	
clip_range	0.2	
entropy_loss	-1.25	
explained_variance	0.0763	
learning_rate	0.0003	
loss	-0.0227	
n_updates	30	
policy_gradient_loss	-0.0573	
value_loss	0.000908	

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).

```
return datetime.utcnow().replace(tzinfo=utc)
```

```

-----
KeyboardInterrupt                                Traceback (most recent call last)
/tmp/ipython-input-4070576608.py in <cell line: 0>()
    59 # Train PPO agent
    60 model = PPO("MlpPolicy", vec_env, verbose=1)
--> 61 model.learn(total_timesteps=10000)
    62
    63 # Evaluate selected features

/usr/local/lib/python3.12/dist-packages/stable_baselines3/ppo/ppo.py in learn(self, total_timesteps, callback, log_interval, tb_log_name, reset_num_timesteps, progress_bar)
    309         progress_bar: bool = False,
    310     ) -> SelfPPO:
--> 311         return super().learn(
    312             total_timesteps=total_timesteps,
    313             callback=callback,

/usr/local/lib/python3.12/dist-packages/stable_baselines3/common/on_policy_algorithm.py in learn(self, total_timesteps, callback, log_interval, tb_log_name, reset_num_timesteps, progress_bar)
    322
    323         while self.num_timesteps < total_timesteps:
--> 324             continue_training = self.collect_rollouts(self.env, callback, self.rollout_buffer, n_rollout_steps=self.n_steps)
    325
    326             if not continue_training:

/usr/local/lib/python3.12/dist-packages/stable_baselines3/common/on_policy_algorithm.py in collect_rollouts(self, env, callback, rollout_buffer, n_rollout_steps)
    216                 clipped_actions = np.clip(actions, self.action_space.low, self.action_space.high)
    217
--> 218                 new_obs, rewards, dones, infos = env.step(clipped_actions)
    219
    220                 self.num_timesteps += env.num_envs

/usr/local/lib/python3.12/dist-packages/stable_baselines3/common/vec_env/base_vec_env.py in step(self, actions)
    220         """
    221         self.step_async(actions)
--> 222         return self.step_wait()
    223
    224     def get_images(self) -> Sequence[Optional[np.ndarray]]:

/usr/local/lib/python3.12/dist-packages/stable_baselines3/common/vec_env/dummy_vec_env.py in step_wait(self)
    57         # Avoid circular imports
    58         for env_idx in range(self.num_envs):
--> 59             obs, self.buf_rews[env_idx], terminated, truncated, self.buf_infos[env_idx] = self.envs[env_idx].step( # type: ignore[assignment]
    60                 self.actions[env_idx]
    61             )

/usr/local/lib/python3.12/dist-packages/shimmy/openai_gym_compatibility.py in step(self, action)
    248         (observation, reward, terminated, truncated, info)
    249         """
--> 250         obs, reward, done, info = self.gym_env.step(action)
    251
    252         if self.render_mode is not None:

/tmp/ipython-input-4070576608.py in step(self, action)
    38         self.current_step += 1
    39         done = self.current_step == self.n_features
--> 40         reward = self.evaluate_model() if done else 0
    41         return self.selected_features, reward, done, {}
    42

/tmp/ipython-input-4070576608.py in evaluate_model(self)
    46         return 0
    47         clf = RandomForestClassifier(n_estimators=100, random_state=42)
--> 48         clf.fit(self.X_train[:, selected_indices], self.y_train)
    49         predictions = clf.predict(X_test[:, selected_indices])
    50         return accuracy_score(y_test, predictions)

/usr/local/lib/python3.12/dist-packages/sklearn/base.py in wrapper(estimator, *args, **kwargs)
   1387         )
   1388         ):
-> 1389         return fit_method(estimator, *args, **kwargs)
   1390
   1391         return wrapper

/usr/local/lib/python3.12/dist-packages/sklearn/ensemble/_forest.py in fit(self, X, y, sample_weight)
    485         # parallel_backend contexts set at a higher level,
    486         # since correctness does not rely on using threads.
--> 487         trees = Parallel(
    488             n_jobs=self.n_jobs,
    489             verbose=self.verbose,

/usr/local/lib/python3.12/dist-packages/sklearn/utils/parallel.py in __call__(self, iterable)
    75         for delayed_func, args, kwargs in iterable
    76         )
--> 77         return super().__call__(iterable_with_config)
    78
    79

/usr/local/lib/python3.12/dist-packages/joblib/parallel.py in __call__(self, iterable)
   1984         output = self._get_sequential_output(iterable)
   1985         next(output)
-> 1986         return output if self.return_generator else list(output)
   1987
   1988         # Let's create an ID that uniquely identifies the current call. If the

/usr/local/lib/python3.12/dist-packages/joblib/parallel.py in _get_sequential_output(self, iterable)
   1912         self.n_dispatched_batches += 1
   1913         self.n_dispatched_tasks += 1
-> 1914         res = func(*args, **kwargs)
   1915         self.n_completed_tasks += 1

```

```
1916         self.print_progress()

/usr/local/lib/python3.12/dist-packages/sklearn/utils/parallel.py in __call__(self, *args, **kwargs)
137         config = {}
138         with config_context(**config):
--> 139             return self.function(*args, **kwargs)
140
141

/usr/local/lib/python3.12/dist-packages/sklearn/ensemble/_forest.py in _parallel_build_trees(tree, bootstrap, X, y, sample_weight, tree_idx, n_trees, verbose,
class_weight, n_samples_bootstrap, missing_values_in_feature_mask)
170         n_samples = X.shape[0]
171         if sample_weight is None:
--> 172             curr_sample_weight = np.ones((n_samples,), dtype=np.float64)
173         else:
174             curr_sample_weight = sample_weight.copy()

/usr/local/lib/python3.12/dist-packages/numpy/core/numeric.py in ones(shape, dtype, order, like)
190
191     a = empty(shape, dtype, order)
--> 192     multiarray.copyto(a, 1, casting='unsafe')
193     return a
194

KeyboardInterrupt:

/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal
in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    return datetime.utcnow().replace(tzinfo=utc)
```

🎮 توضیحات مثال کد پایتون

در این مثال، از **یادگیری تقویتی (Reinforcement Learning)** برای انتخاب ویژگی‌های مؤثر در طبقه‌بندی داده‌های آیریس استفاده شده است. به جای الگوریتم‌های سنتی، یک عامل هوشمند با استفاده از الگوریتم **PPO** در محیط **Gym** آموزش دیده تا تصمیم بگیرد کدام ویژگی‌ها را انتخاب کند:

- ابتدا داده‌های آیریس بارگذاری و به دو بخش آموزش و آزمون تقسیم شده‌اند.
- یک محیط سفارشی **FeatureSelectionEnv** تعریف شده که در آن عامل در هر مرحله یک ویژگی را انتخاب می‌کند.
- پاداش عامل فقط زمانی داده می‌شود که همه ویژگی‌ها انتخاب شده باشند و مدل **Random Forest** روی ویژگی‌های انتخاب‌شده آموزش داده شود.
- عامل با استفاده از الگوریتم **Proximal Policy Optimization (PPO)** آموزش دیده تا یاد بگیرد کدام ویژگی‌ها منجر به بیشترین دقت طبقه‌بندی می‌شوند.
- در پایان، عامل به صورت greedy ویژگی‌هایی را انتخاب کرده و اندیس آن‌ها نمایش داده شده است.

این مثال نشان می‌دهد که چگونه می‌توان از **یادگیری تقویتی** برای حل مسائل انتخاب ویژگی استفاده کرد. چنین رویکردهایی در مسائل پیچیده با فضای جستجوی بزرگ می‌توانند عملکرد بهتری نسبت به روش‌های کلاسیک داشته باشند.

The End of Chapter 7